# [BUSINESS] FORECASTING AND PREDICTIVE MODELING

## LECTURE 6

Lect. PhD. Oneț-Marian Zsuzsanna

Babeș - Bolyai University
Computer Science and Mathematics Faculty

2025 - 2026

# In Lecture 5 ...

- Time series transformations
- Time series decomposition

# Today

- Time series decomposition
- Time series features

# Time series decomposition - Recap

- The goal of time series decomposition is to split a time series into its components:
  - trend (or trend-cycle) - $T$,
  - seasonality - $S$
  - remainder - $R$.
- We can assume additive decomposition:

$$y_t = S_t + T_t + R_t$$

where $y$ are the data, $S$ is the seasonal component, $T$ is the trend component, and $R$ is the remainder, all in period $t$.

- Or, we can assume a multiplicative decomposition:

$$y_t = S_t * T_t * R_t$$

# History of time series decomposition models

- The classical decomposition was developed in 1920.
- In 1957 the Census II method was introduced, which was the basis for the X-11 method and its variants (X-12-ARIMA and X-13-ARIMA)
- STL method was introduced in 1983
- SEATS and TRAMO were introduced in the 1990s.

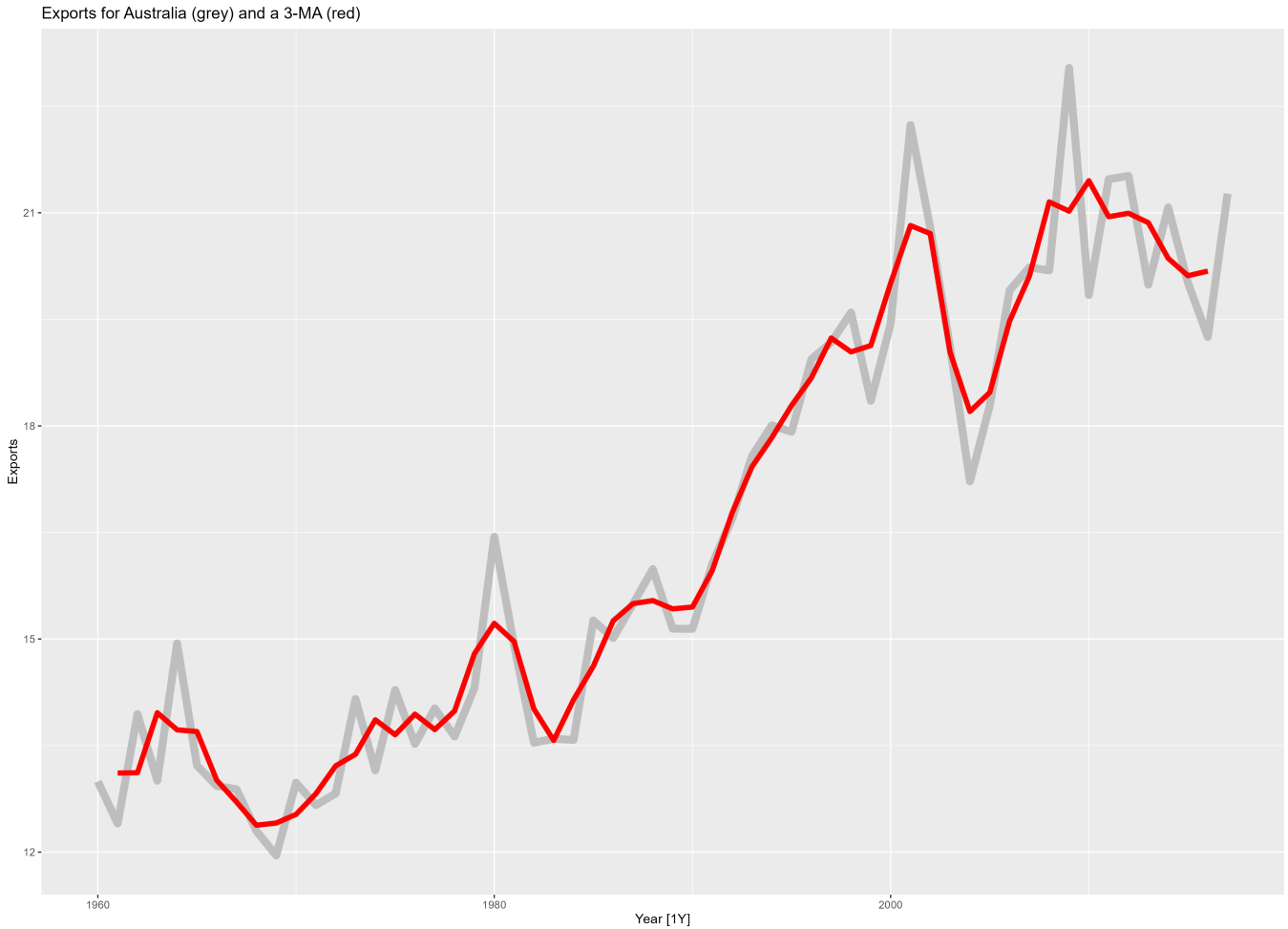
# Moving averages - Recap

- Moving averages are needed in order to estimate the trend-cycle component in the classical decomposition.
- A moving average of order $m$, denoted by $m$-**MA** can be computed as:

$$\hat{T}_t = \frac{1}{m} \sum_{j=-k}^{k} y_{t+j}$$

- where $m = 2k + 1$.
- In short, for estimating the moving average at a given time $t$, we compute the average of that value and the values for the previous and the next $k$ observations. Assuming that observations close in time are close in value as well, the moving average will eliminate some of the randomness (will generate a smoother series).

- 3-MA and original data



# Classical decomposition method I

- Assuming that we have a time series, with a seasonal period of m the classical additive decomposition (where we assume $y_t = S_t + T_t + R_t$ has the following steps:

  1. If $m$ is even, compute the $2xm - MA$, otherwise compute the $m - MA$ of the series. This will be the *trend-cycle* component, $\hat{T}_t$.

  2. Compute the detrended series: $y_t - \hat{T}_t$

# Classical decomposition method II

3. Estimate the seasonal component, by computing for every season the average of the detrended values for that season (for example, for quarterly data, compute the average of all Q1 values, the average of all Q2 values, etc.). Adjust the seasonal components to make sure they add up to zero. The seasonal component, $\hat{S}_t$ is the sequence of these values, replicated for each year of the data.

4. Compute the remainder, by removing the trend-cycle and seasonal components from the data: $\hat{R}_t = y_t - \hat{T}_t - \hat{S}_t$

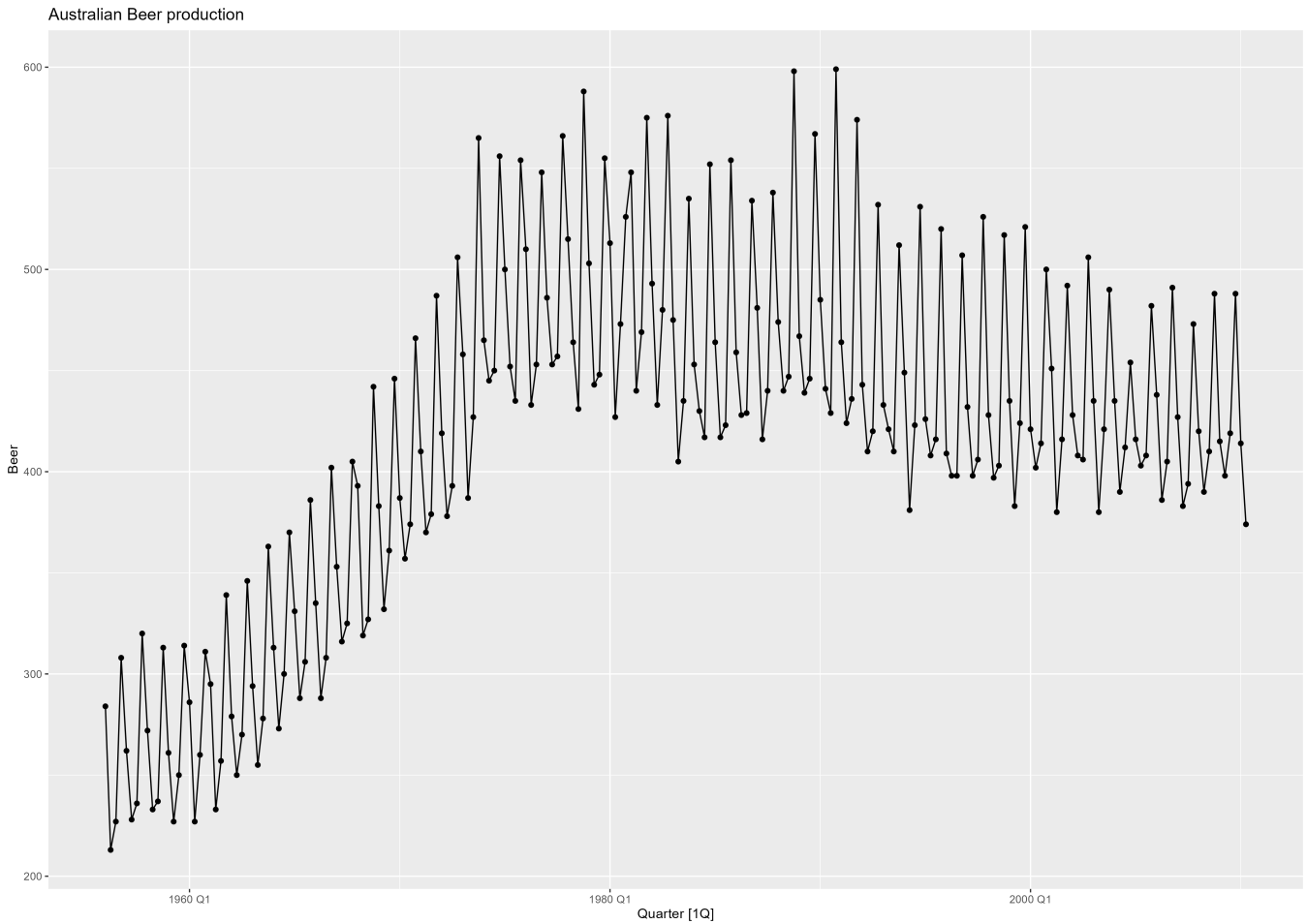
- **Obs.** Multiplicative decomposition is very similar, with the observation that we use division instead of subtraction.

- We have the classical decomposition implemented in R, and it takes a parameter called *type*, which can be "additive" or "multiplicative", to describe what type of decomposition to use.

- From the components of the decomposition, we can see that the remainder is here called *random*, but it represents the same concept: what is left when you eliminate the trend and the season.

```
beer <- aus_production |> select(Beer)
beer |> autoplot()

beerDecomp <- beer |>
  model(d = classical_decomposition(Beer, type = "additive"))

beerDecomp |> components()
beerDecomp |> components() |> autoplot()

beerDecomp |> components() |>
  ACF(random) |> autoplot()
```

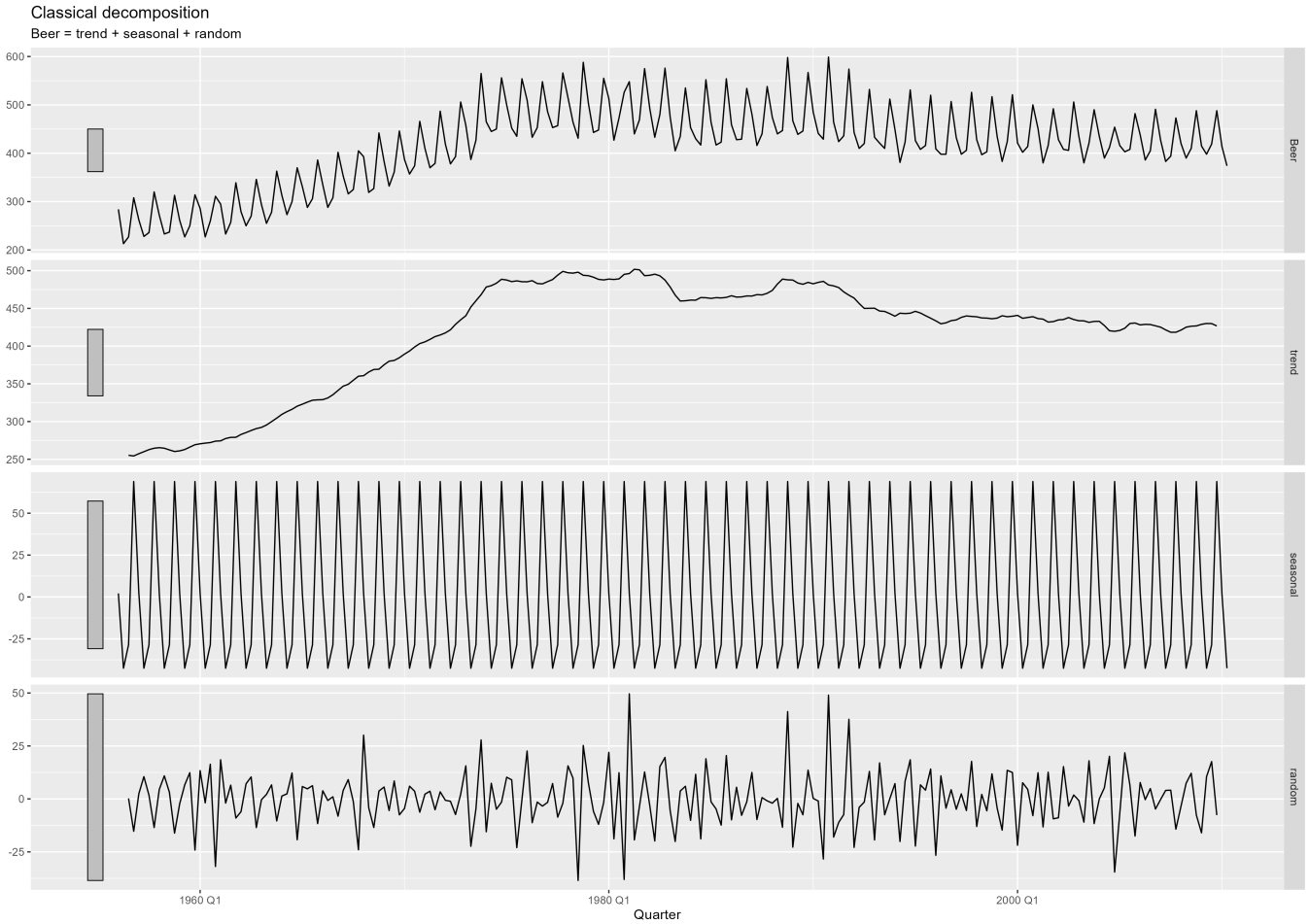
- Initial data



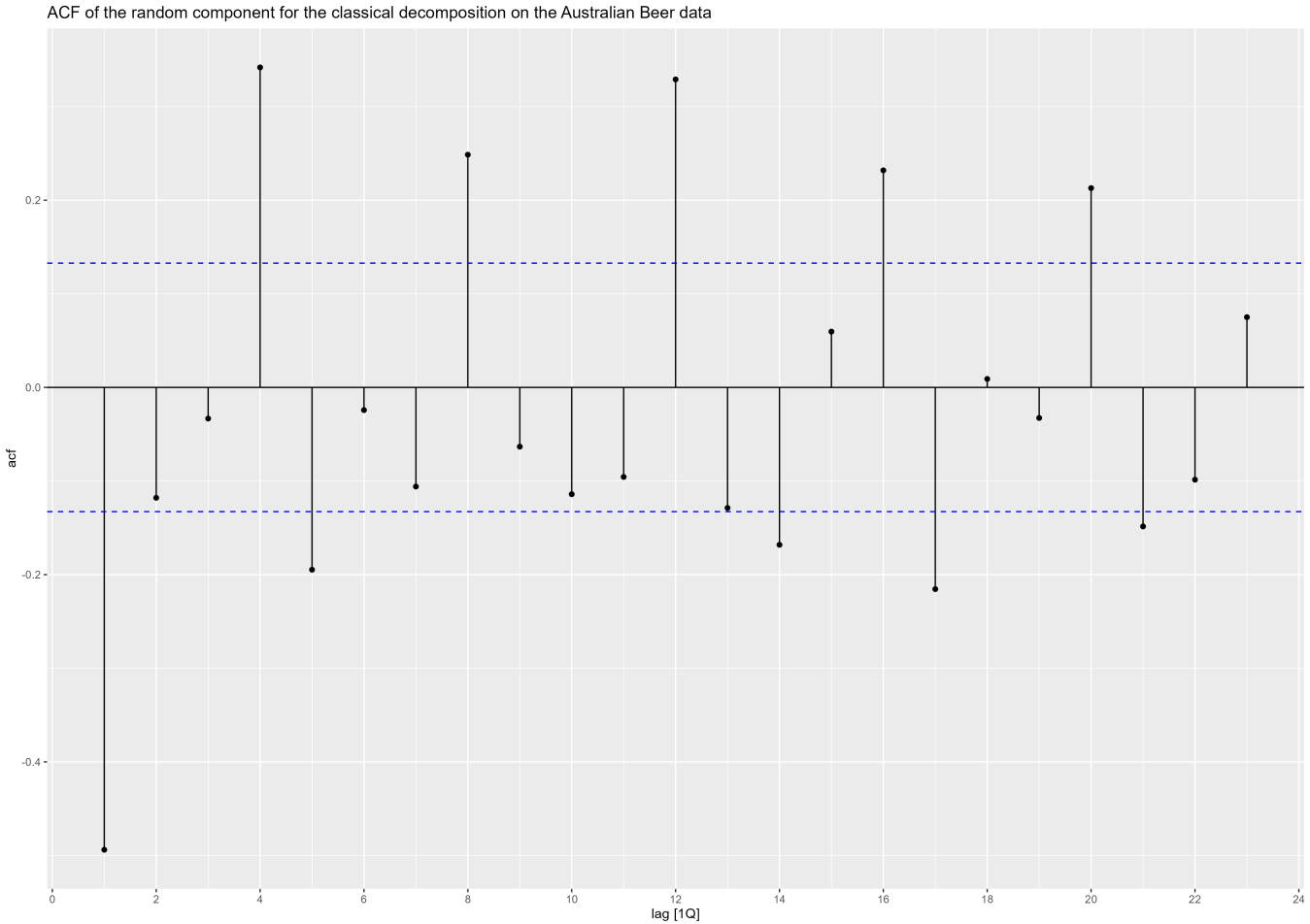
- Components of the classical decomposition

```
# A dable: 218 x 7 [1Q]
# Key:     .model [1]
# :        Beer = trend + seasonal + random
   .model Quarter  Beer trend seasonal  random season_adjust
   <chr>    <qtr> <dbl> <dbl>    <dbl>   <dbl>          <dbl>
 1 d      1956 Q1   284    NA     2.13  NA              282.
 2 d      1956 Q2   213    NA   -42.5   NA              256.
 3 d      1956 Q3   227  255.   -28.5    0.256          256.
 4 d      1956 Q4   308  254.    68.9  -15.3            239.
 5 d      1957 Q1   262  257.     2.13   2.49           260.
 6 d      1957 Q2   228  260    -42.5   10.5            271.
 7 d      1957 Q3   236  263.   -28.5    1.76           265.
 8 d      1957 Q4   320  265.    68.9  -13.5            251.
 9 d      1958 Q1   272  265.     2.13   4.49           270.
10 d      1958 Q2   233  265.   -42.5   10.9            276.
# i 208 more rows
```

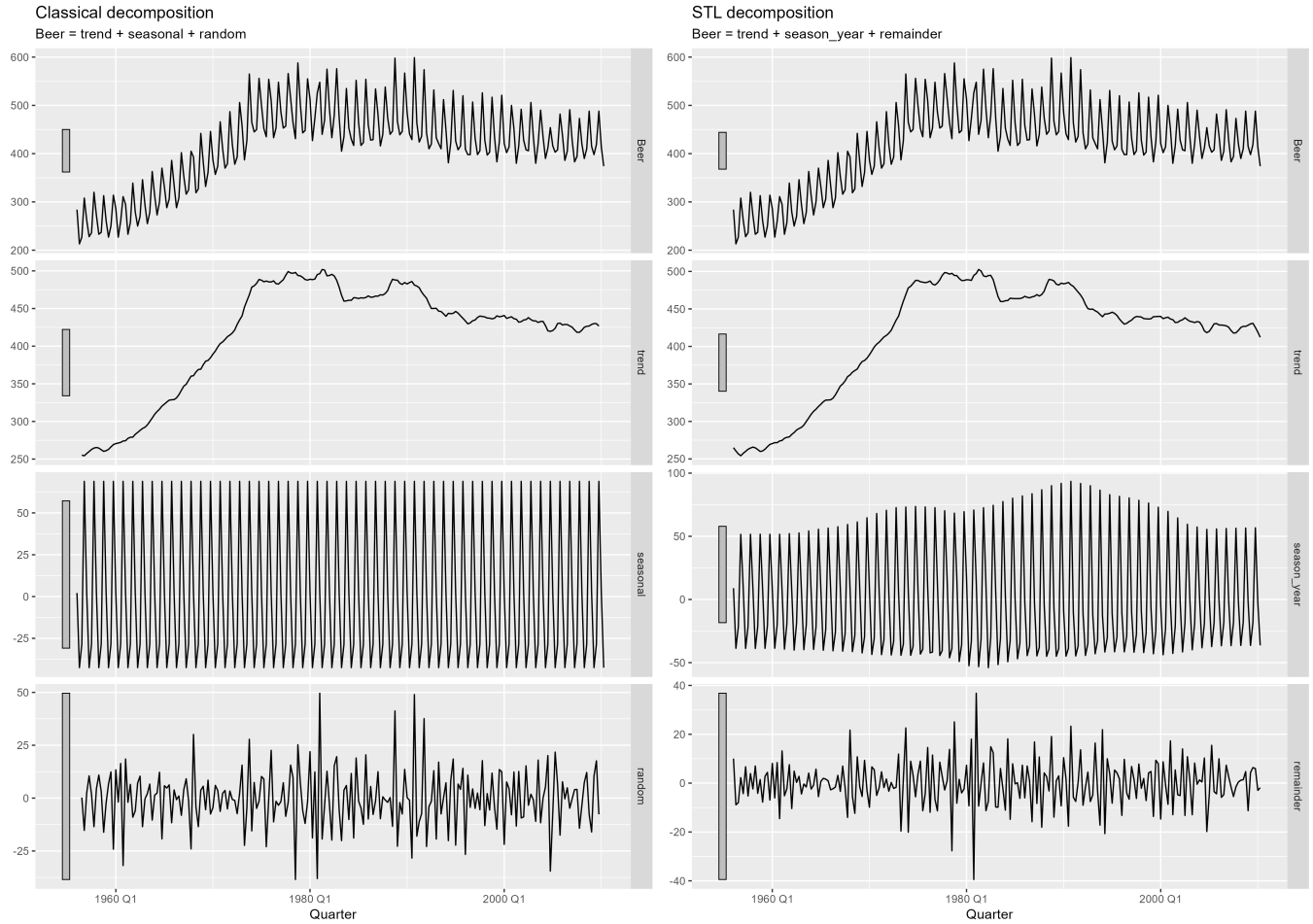
- Plot of the components of the classical decomposition



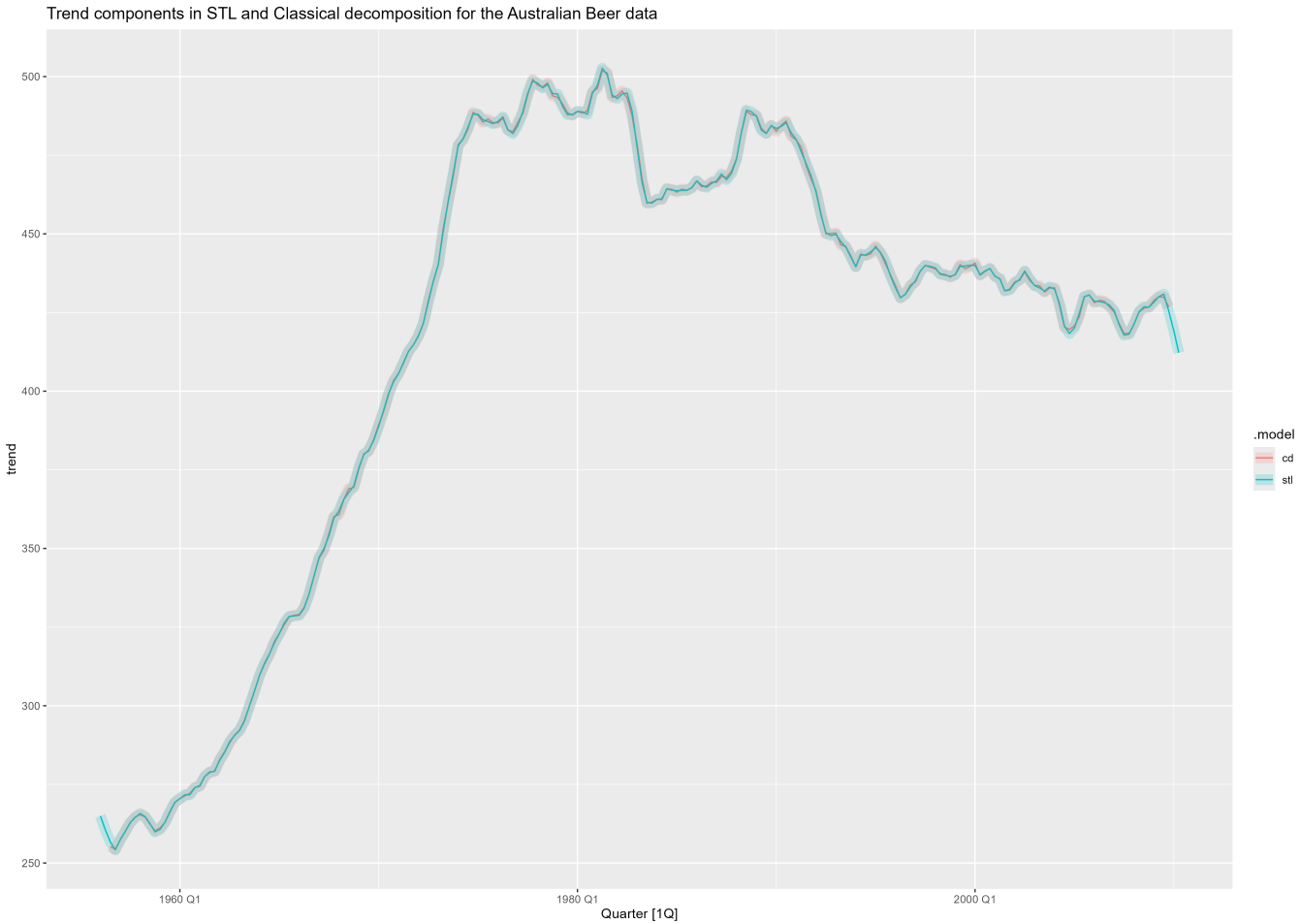
- ACF of the random component to see how much it resembles white noise



- Let's compare the plots of the classical decomposition and STL decomposition. What do you notice?



- Let's look at only the trend component.

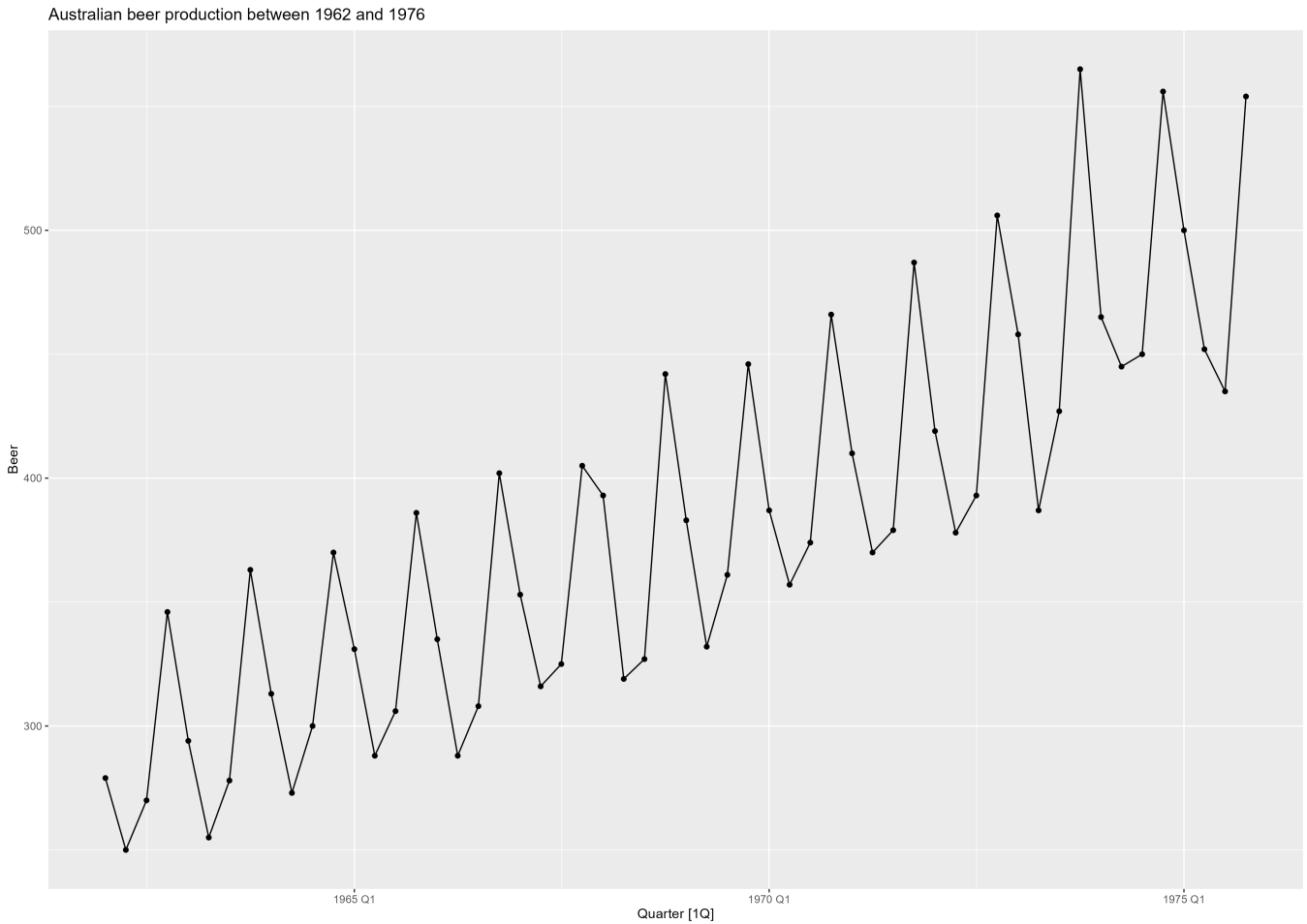


- The ACF plot tells us, that the classical decomposition did not perform well on this time series. One possible reason for such a bad performance is that this is a long time series (218 observations, over 54 years worth of data) and the classical decomposition creates the same seasonal component for the entire period.

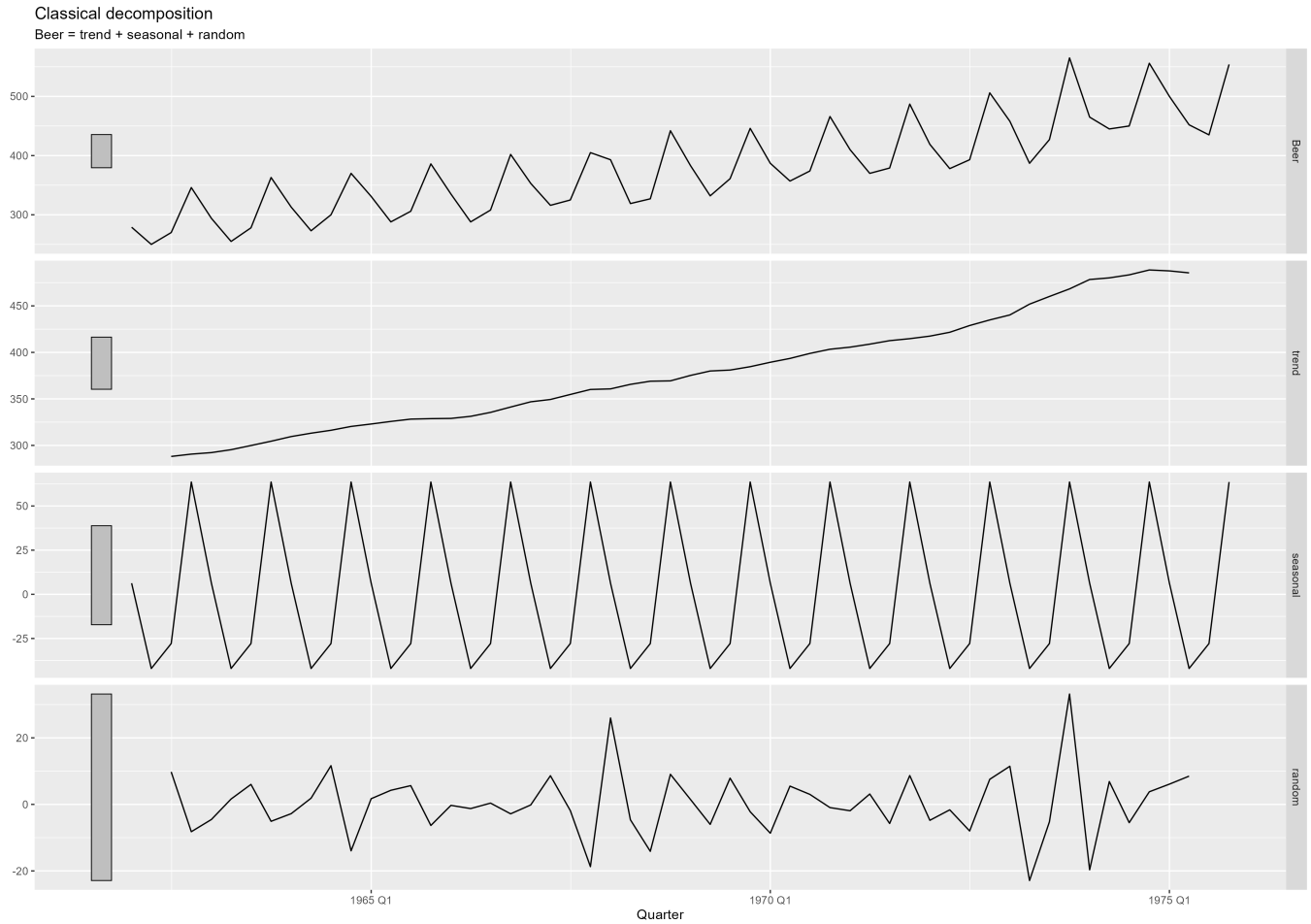
- Let us look at a shorter period, for example between 1962 and 1975.

```
beerShort <- beer |>
  filter(year(Quarter) >= 1962, year(Quarter)< 1976)

beerShort |> autoplot()
beerShortDecomp <- beerShort |>
  model(d = classical_decomposition(Beer, type = "additive"))

beerShortDecomp |> components() |> autoplot()
beerShortDecomp |> components() |>
  ACF(random) |> autoplot()
```

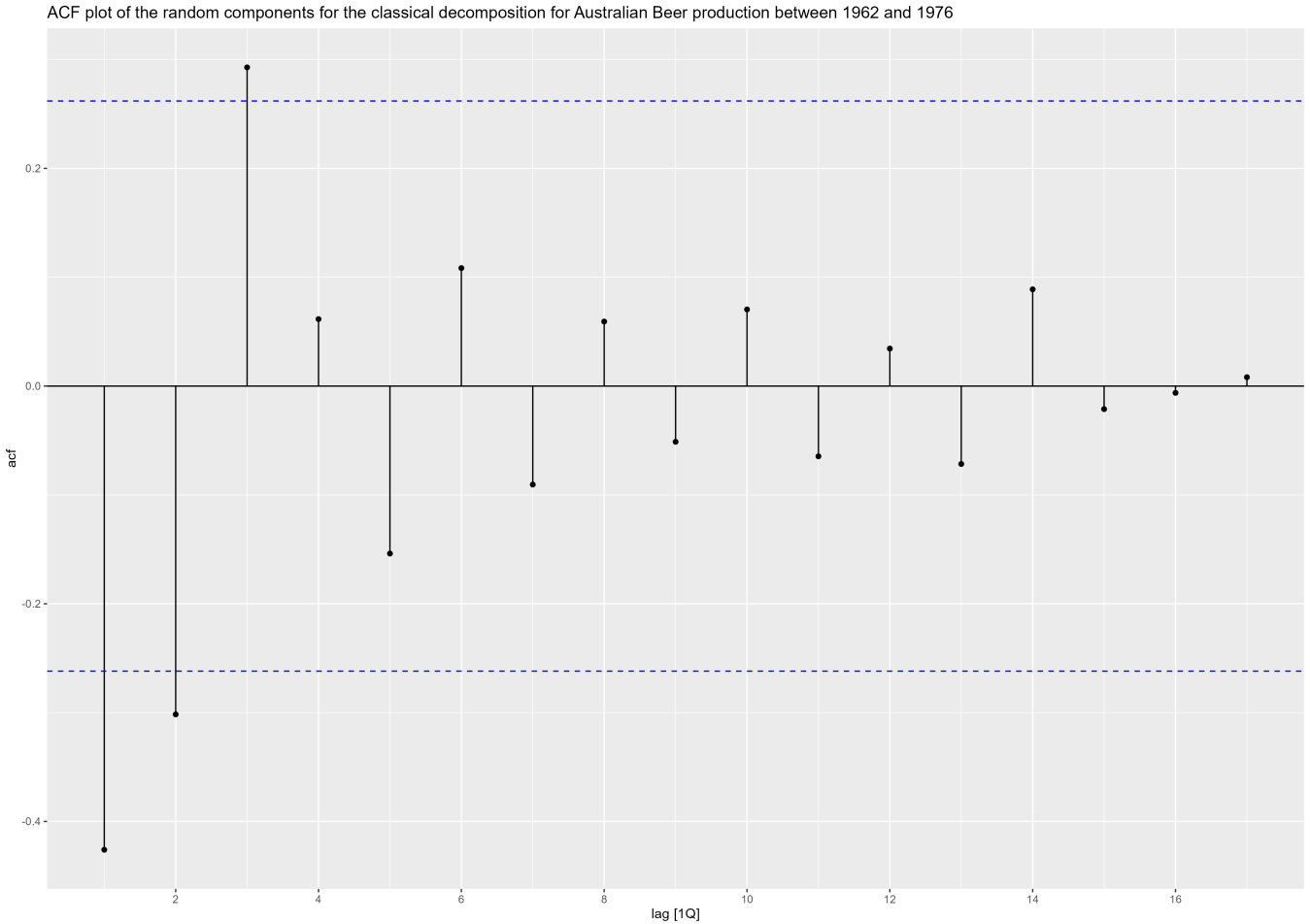
- Initial data between 1962 and 1975



- Components of the classical decomposition



- ACF of the random component

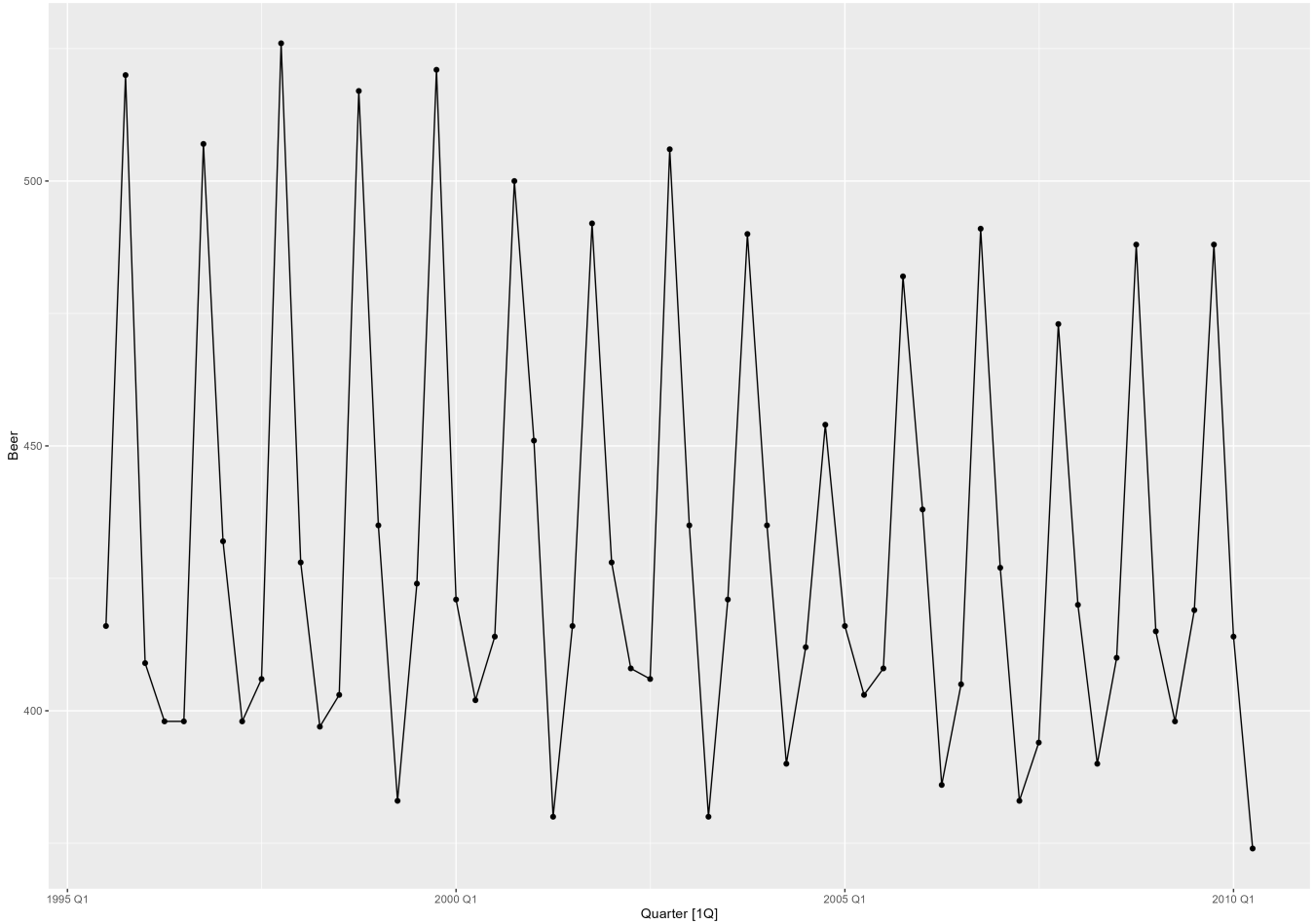



- Still not white noise, but a lot better

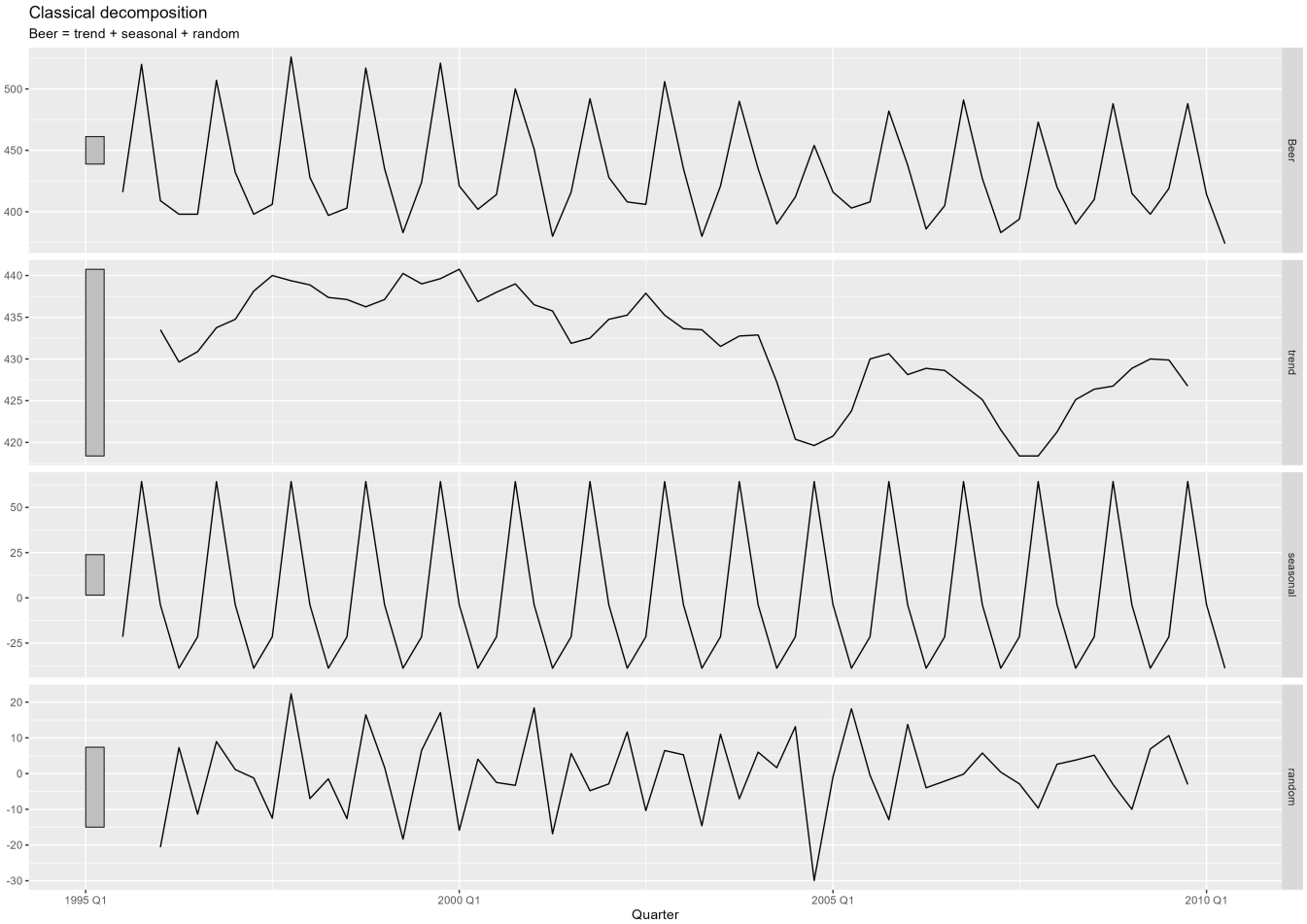
- I choose that part of the data, because the seasonal component seemed pretty uniform there, but normally, we are more interested in recent data, so let us try the same classical decomposition on the last 60 observations as well.

```
beerRecent <- beer |> tail(n = 6)
beerRecent |> autoplot() + geom_point()

beerRecentDecomp <- beerRecent |>
  model(d = classical_decomposition(Beer, type = "additive"))

beerRecentDecomp |> components() |> autoplot()

beerRecentDecomp |> components() |>
  ACF(random) |> autoplot()
```

- Initial data for the observations

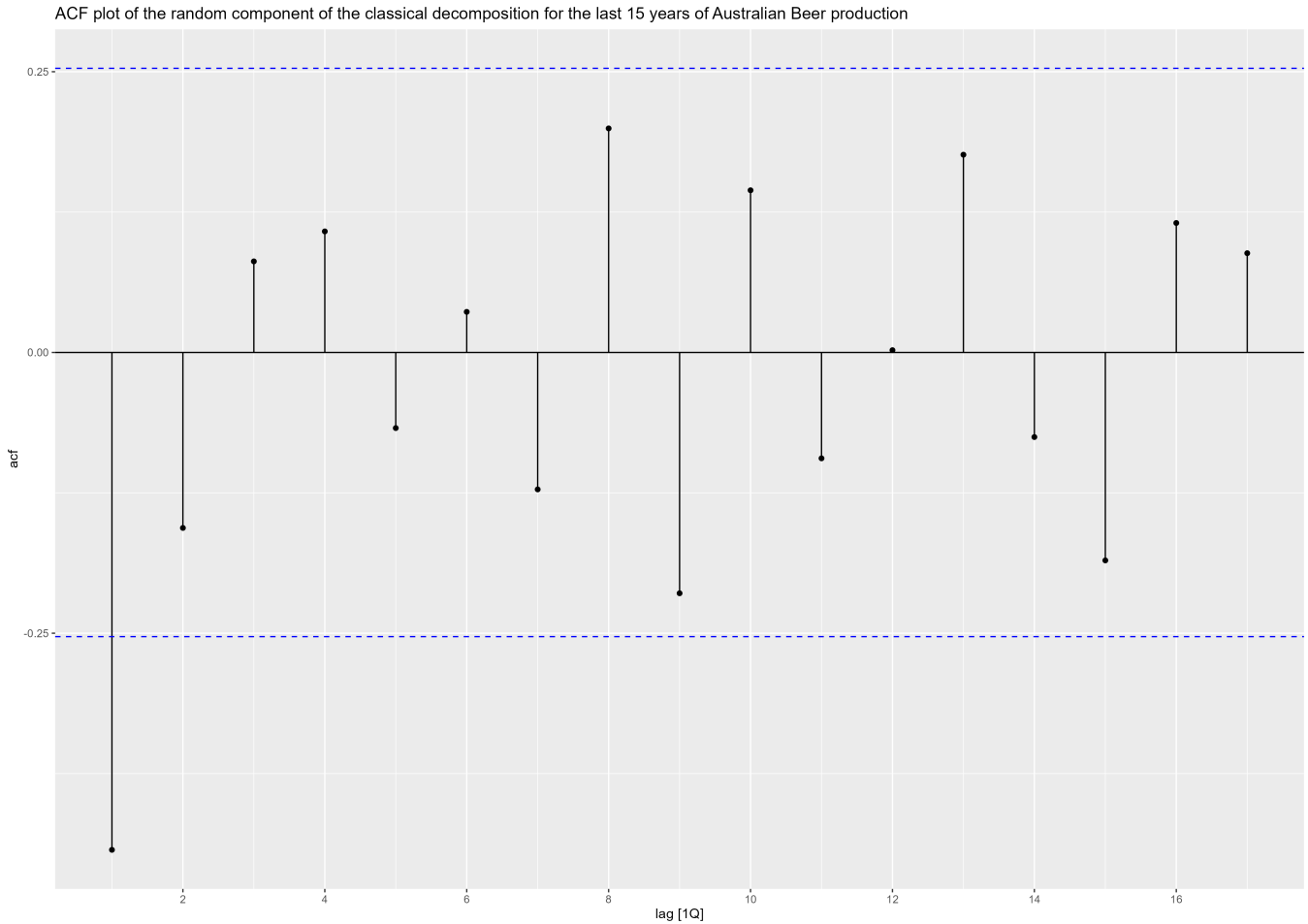


- Components of the classical decomposition




- What do you observe on this plot? Which component looks a little strange?

- ACF of the random component

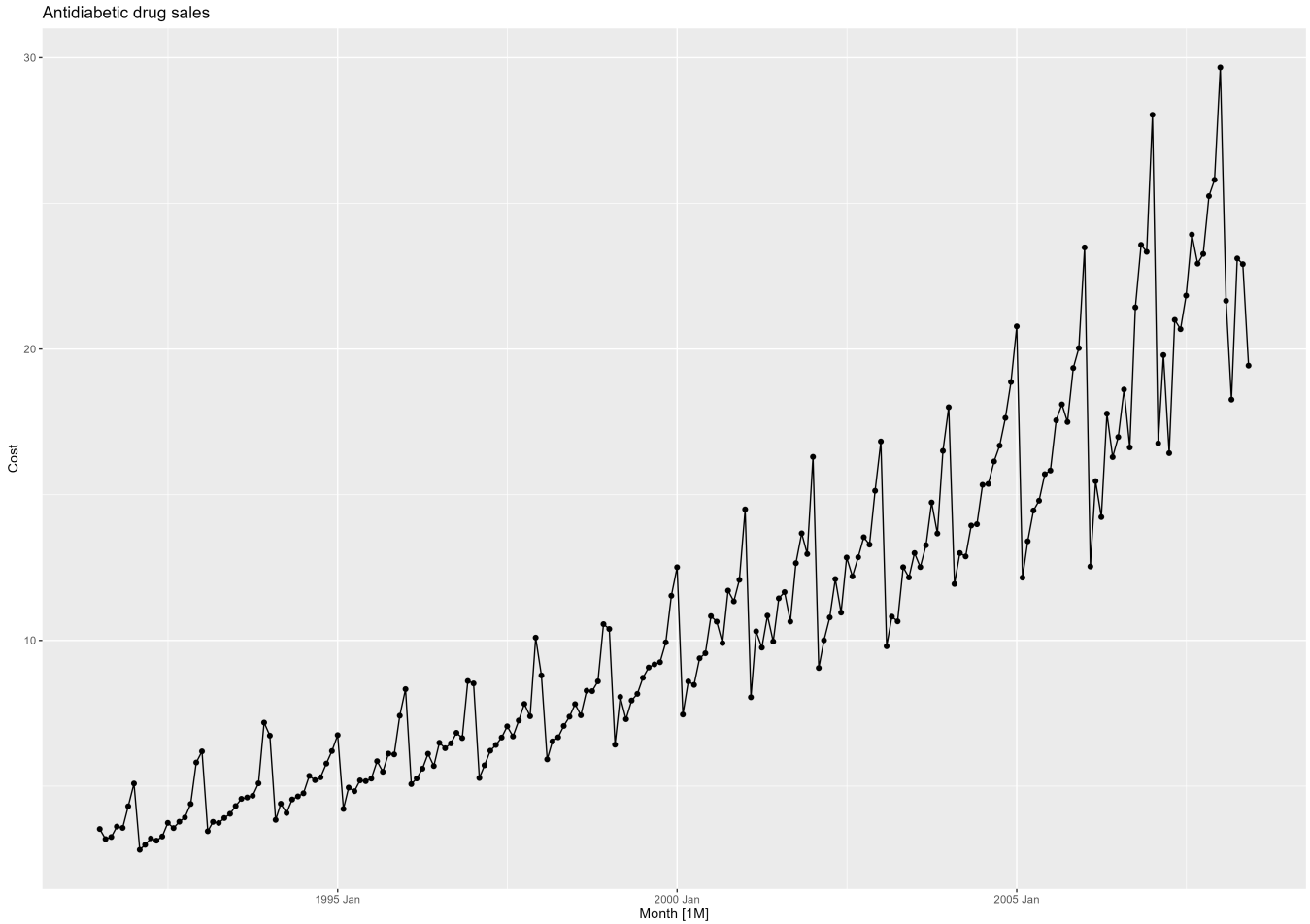


# Multiplicative decomposition

- Let us look at an example where a multiplicative decomposition is more suitable.

```
a10 <- PBS |>
  filter(ATC2 == "A10") |>
  select(Month, Concession, Type, Cost) |>
  summarize(TotalCost = sum(Cost)) |>
  mutate(Cost = TotalCost / 1000000)

a10 |> autoplot(Cost) + geom_point()
a10Decomp <- a10 |> model(d = classical_decomposition(Cost, type = "
    multiplicative"))
a10Decomp |> components()

a10Decomp |> components() |> autoplot()
a10Decomp |> components() |>
  ACF(random) |> autoplot()
```

- Initial data

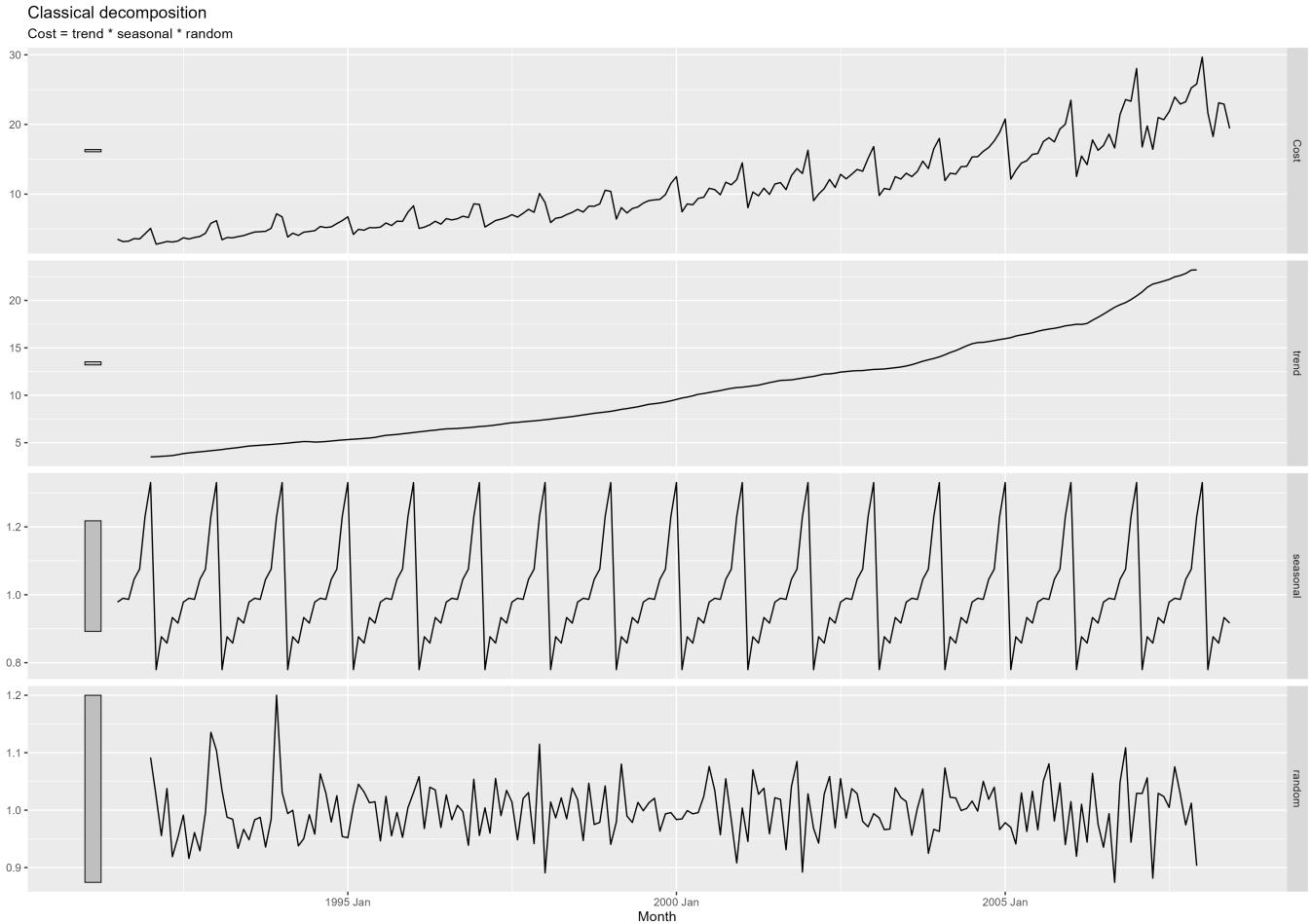


- Components of the classical decomposition

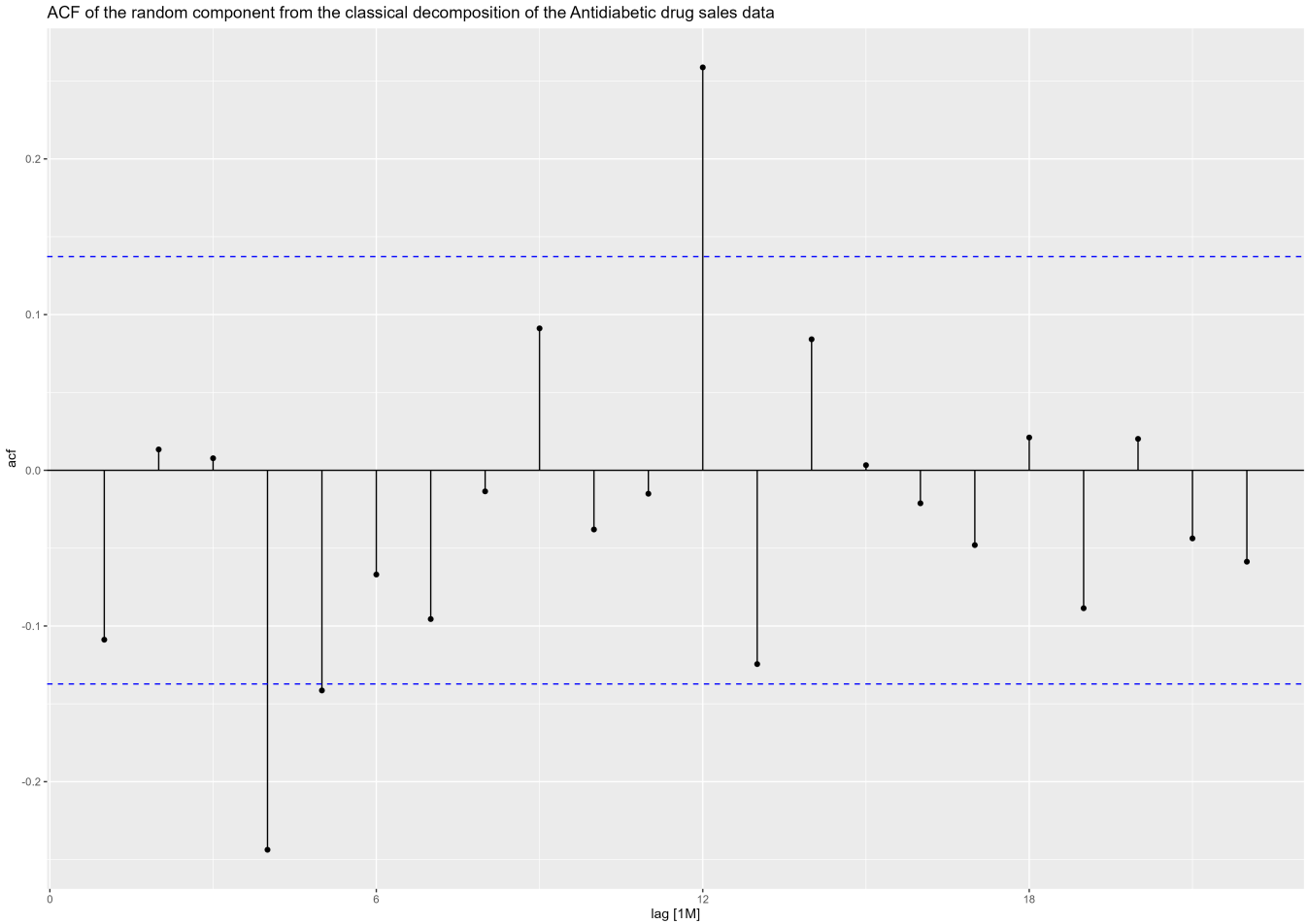
```
# A dable: 204 x 7 [1M]
# Key:     .model [1]
# :        Cost = trend * seasonal * random
   .model    Month  Cost trend seasonal random season_adjust
   <chr>     <mth> <dbl> <dbl>    <dbl>  <dbl>          <dbl>
 1 d      1991 Jul  3.53 NA       0.979 NA              3.60
 2 d      1991 Aug  3.18 NA       0.990 NA              3.21
 3 d      1991 Sep  3.25 NA       0.986 NA              3.30
 4 d      1991 Oct  3.61 NA       1.05  NA              3.45
 5 d      1991 Nov  3.57 NA       1.08  NA              3.32
 6 d      1991 Dec  4.31 NA       1.23  NA              3.50
 7 d      1992 Jan  5.09  3.50    1.33   1.09           3.82
 8 d      1992 Feb  2.81  3.53    0.780  1.02           3.61
 9 d      1992 Mar  2.99  3.57    0.876  0.956          3.41
10 d      1992 Apr  3.20  3.60    0.858  1.04           3.74
# i 194 more rows
# i Use `print(n = ...)` to see more rows
```

- What do you notice in this table?

- Plot of components of the classical decomposition



- ACF of the random component



- Some problems/disadvantages of the classical decomposition method:
  - You do not have trend-cycle estimates for the first and last few observations (since the moving average is not computed for these).
  - The trend-cycle tends to over-smooth rapid rises and falls in data.
  - It cannot capture if seasonal patterns change over time (it just repeats the same sequence of data)
  - Not robust for unusual data (for example: changes in monthly air passenger traffic due to a strike).

- In practice, it is not recommended to use the classical decomposition anymore, since there are better alternatives.

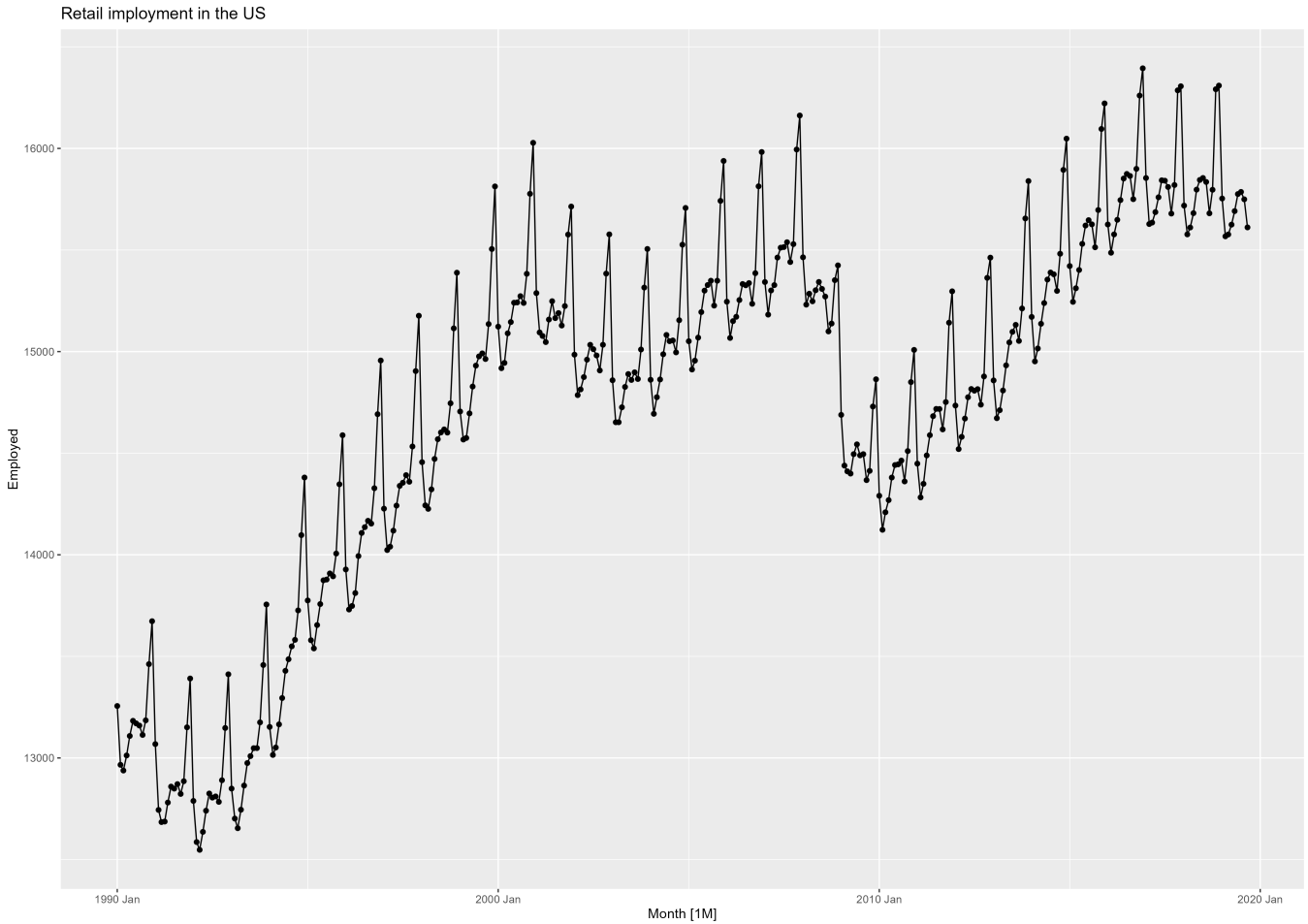
# X-11 decomposition I

- Originated from the US Census Bureau and Statistics Canada.
- It is based on the classical decomposition, but has many extra steps to overcome the drawbacks of classical decomposition:
  - Trend-cycle estimates are available for all data points
  - The seasonal component is allowed to vary slowly over time
  - It can handle holiday effects and trading day variation (the fact that not all months have the same number of working days)
- It can do both multiplicative and additive decomposition
- The model is automatic and robust to outliers
- As a disadvantage, it can only work with monthly and quarterly data.

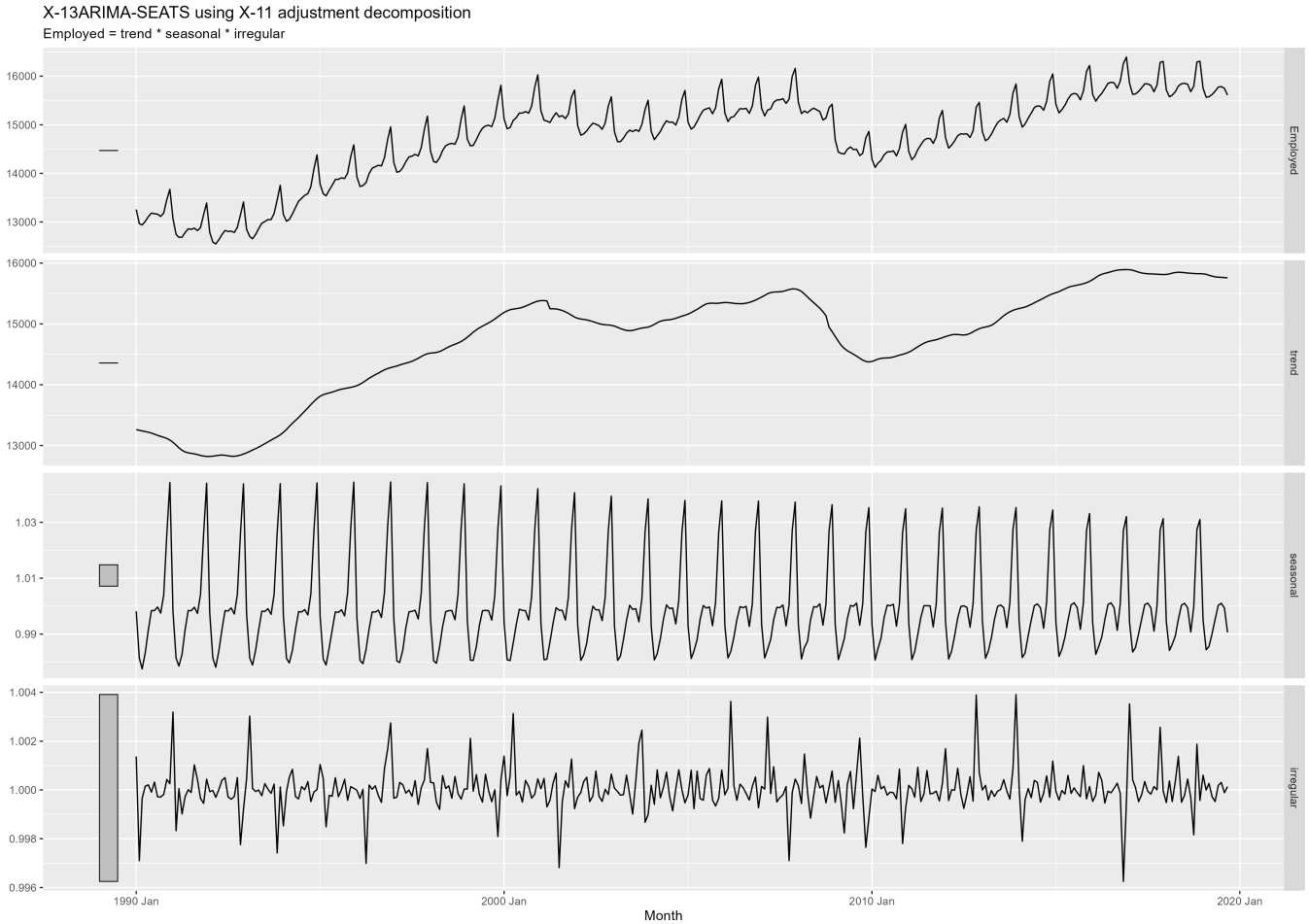
# X-11 decomposition II

- In R, it can be found as part of the *X_13ARIMA_SEATS* group of methods (so, you create an *X_13ARIMA_SEATS* model, and then you specify as parameter that you want to use the *x11* method)

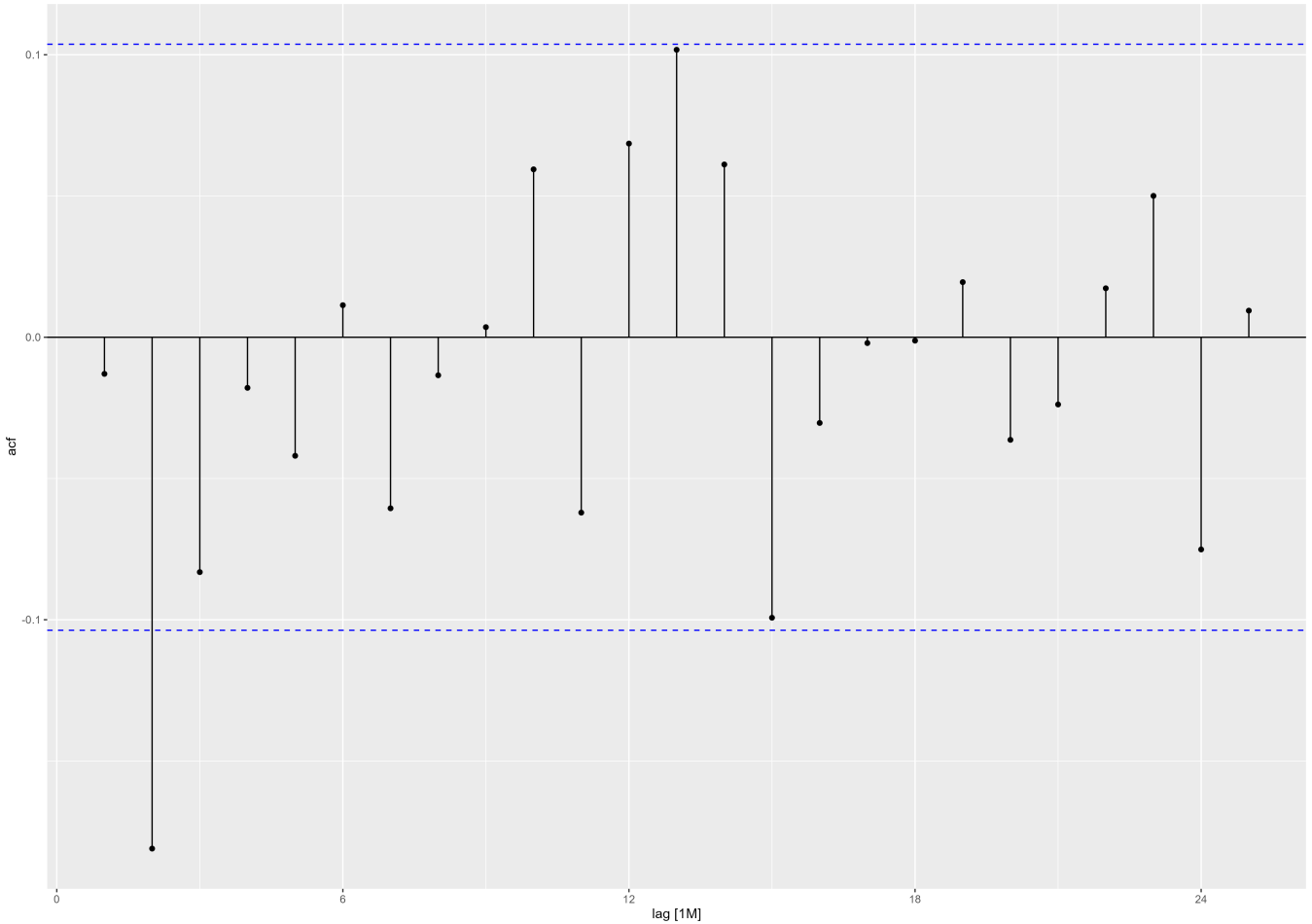
```
x_11_decomp <- us_retail_employment |>
  model(x11 = X_13ARIMA_SEATS(Employed ~ x11()))

x_11_decomp |> components() |> autoplot()

x_11_decomp |> components() |> tail(n = 10)

x_11_decomp |>
   components() |>
  ACF(irregular) |>
  autoplot() +
  geom_point() +
  labs("ACF of the X11 decomposition on the Us retail employment data")
```


Retail imployment in the US
16000
15000
14000
13000
Employed
1990 Jan
2000 Jan
2010 Jan
2020 Jan
Month [1M]

```
# A dable: 10 x 7 [1M]
# Key:     .model [1]
# :        Employed = trend * seasonal * irregular
   .model    Month Employed  trend seasonal irregular season_adjust
   <chr>     <mth>    <dbl>  <dbl>    <dbl>      <dbl>          <dbl>
 1 x11    2018 Dec   16309. 15826.    1.03       1.00         15819.
 2 x11    2019 Jan   15754. 15823.    0.995      1.00         15833.
 3 x11    2019 Feb   15567. 15815.    0.984      1.00         15815.
 4 x11    2019 Mar   15577. 15800.    0.986      1.00         15805.
 5 x11    2019 Apr   15625. 15785.    0.990      1.00         15780.
 6 x11    2019 May   15692. 15773.    0.995      1.00         15766.
 7 x11    2019 Jun   15776. 15768.    1.00       1.00         15771.
 8 x11    2019 Jul   15786. 15765.    1.00       1.00         15770.
 9 x11    2019 Aug   15750. 15761.    0.999      1.00         15760.
10 x11    2019 Sep   15611. 15757.    0.991      1.00         15759.
```

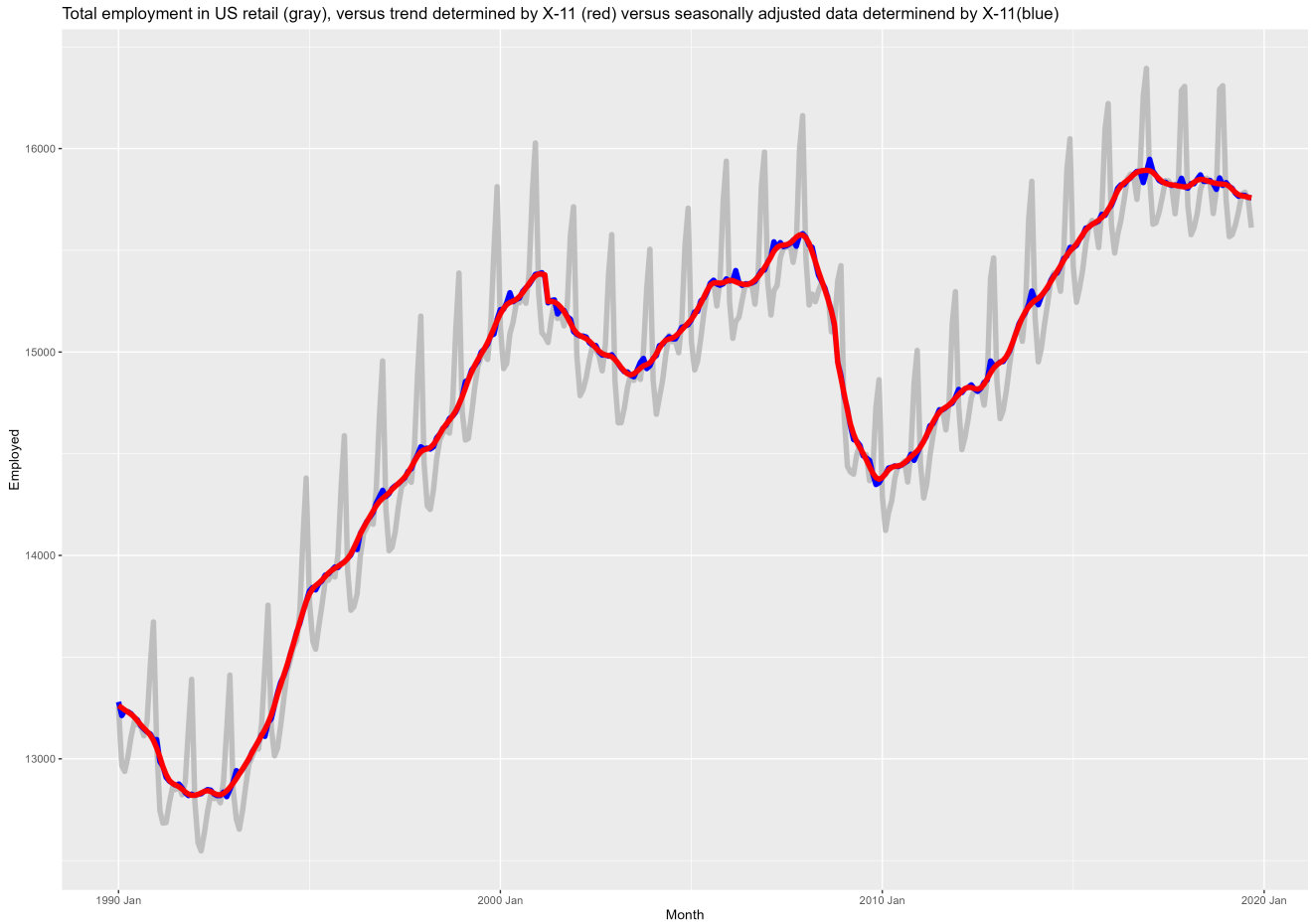



- What do you notice on the plot?


0.1
0.0
-0.1
acf
0
6
12
18
24
lag [1M]

- If we plot the trend and the seasonally adjusted data (which is trend + remainder) together, we can see how well the trend covers the data

```
x_11_decomp |>
  components() |>
  ggplot(aes(x = Month)) +
  geom_line(aes(y = Employed ), colour = "gray", size = 2) +
  geom_line(aes(y = season_adjust), colour = "blue", size = 2) +
  geom_line(aes(y = trend), colour = "red", size = 2) +
  labs(title = "Total employment in US retail (gray), versus trend determined by
      X-11 (red) versus seasonally adjusted data determinend by X-11(blue)")
```


Total employment in US retail (gray), versus trend determined by X-11 (red) versus seasonally adjusted data determinend by X-11(blue)
16000
15000
14000
13000
Employed
1990 Jan
2000 Jan
2010 Jan
2020 Jan
Month

- Some details about the implementation of X11 can be found on the webpage of the *Australian Bureau of Statistics*: `https://www.abs.gov.au/websitedbs/d3310114.nsf/4a256353001af3ed4b2562bb00121564/c890aa8e65957397ca256ce10018c9d8!OpenDocument`

- According to the above document, the X11 algorithm is the following, assuming monthly data and multiplicative decompositions:

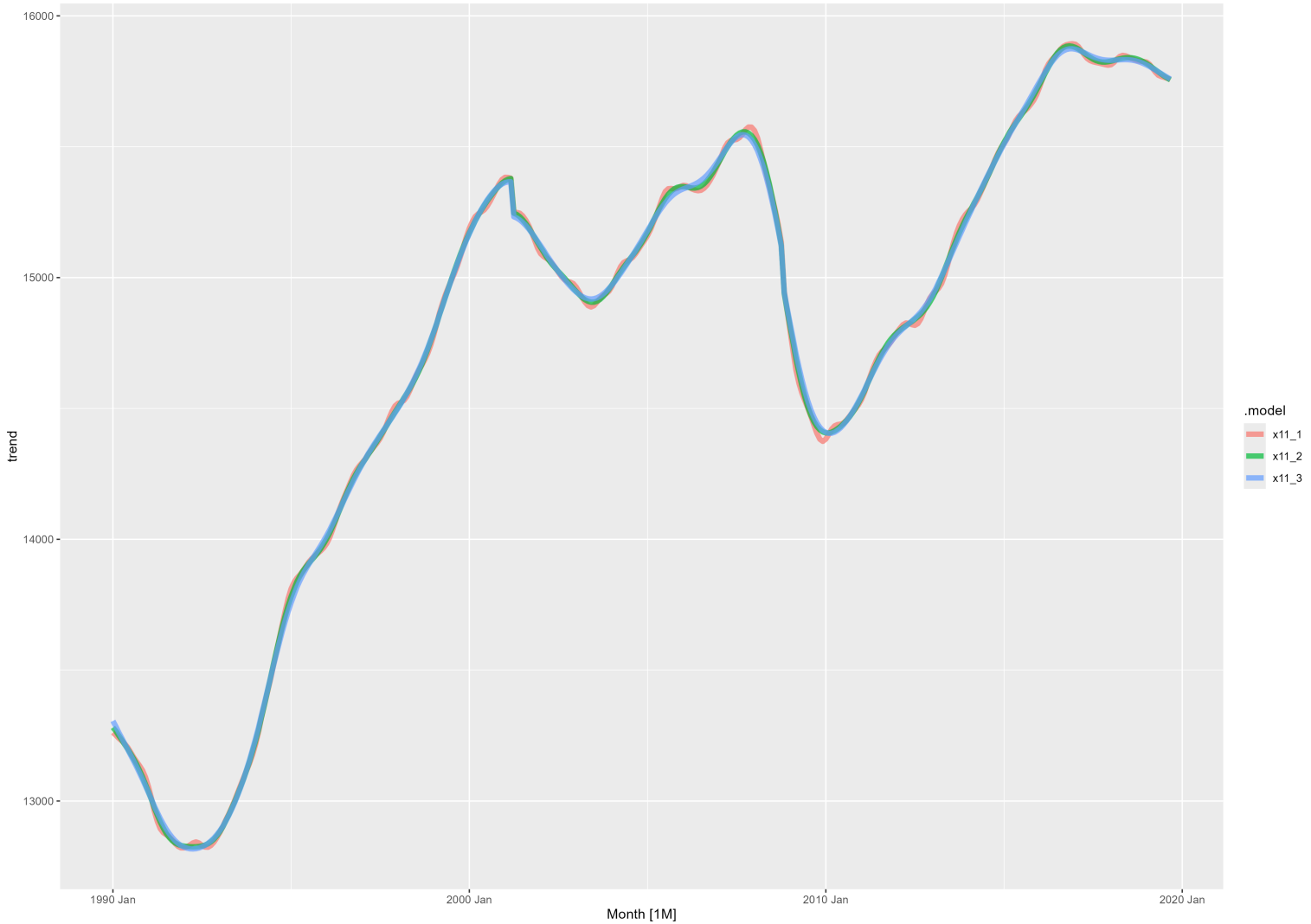
1. Use a 2x12-MA to create an initial estimate of the trend component. 6 values at both ends will not have trend values.
2. Remove the trend from the series (by dividing by it), leaving only the seasonal and irregular/random/remainder components.
3. Build a preliminary estimate of the seasonal component, by applying a 3x3-MA on the detrended season for each month separately. Missing values at the ends are replaced by repeating the previous year's pattern.
4. Compute the seasonally adjusted time series, by dividing the original one by the seasonal component.
5. Re-estimate the trend, by applying a 9, 13 or 23 term Henderson moving average to the seasonally adjusted values. Use asymetric filters at the end to have an estimate for every time step.
6. Divide the original time series by this new estimate of trend and re-estimate the season again (repeat step 3).

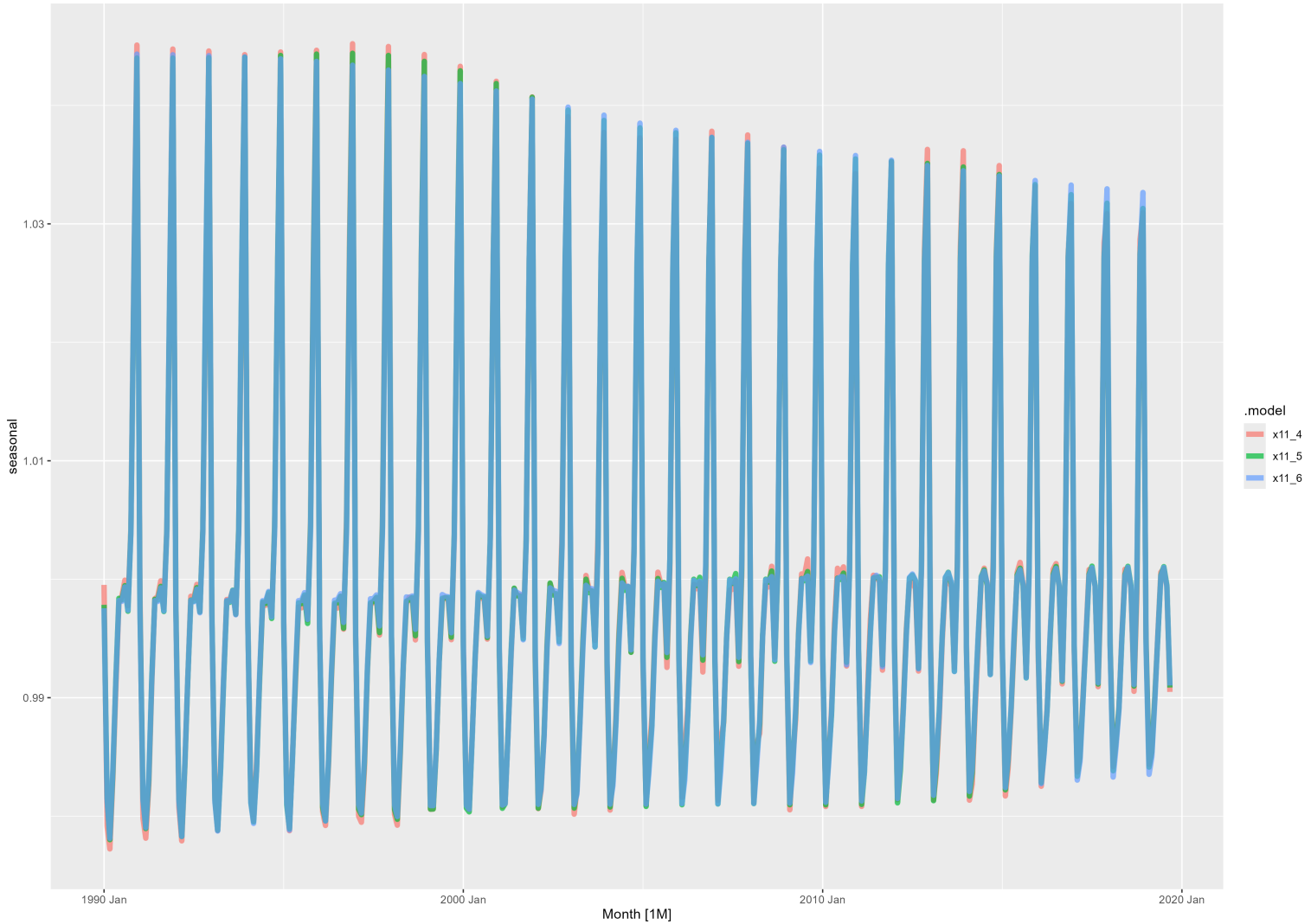
7. Final estimate of seasonally adjusted data is computed by dividing the original time series by this new seasonal component.
8. Compute the final estimate of the trend component by applying a 9, 13 or 23 term Henderson moving average.
9. Compute the final estimate of the irregular component, by diving the time series by the trend and the seasonal component.

- The Henderson moving average is a weighted, symmetric moving average, which works with special, pre-defined weights. For example, for the 13 term Henderson moving average we have the following weights:
  - -0.02, -0.03, 0, 0.07, 0.15, 0.21, 0.24, 0.21, 0.15, 0.07, 0.00, -0.03, -0.02

- **Obs:** In the R implementation, you can specify the moving averages for the trend and the seasonal component estimation.
  - For trend, use the *trendma* parameter, whose value needs to be a positive odd integer.
  - For season use the *seasonalma* parameter, whose value needs to be "s3x1", "s3x3", "s3x5", "s3x9", "s3x15", "stable", "msr", or "default"
- What do you think is the difference between using "s3x1" and "s3x15" for the seasonal component?

```
x_11_decomps <- us_retail_employment |>
  model(x11_1 = X_13ARIMA_SEATS(Employed ~ x11(trendma = 9)),
  x11_2 = X_13ARIMA_SEATS(Employed ~ x11(trendma = 27)),
  x11_3 = X_13ARIMA_SEATS(Employed ~ x11(trendma = 39)),
  x11_4 = X_13ARIMA_SEATS(Employed ~ x11(seasonalma = "s3x1")),
  x11_5 = X_13ARIMA_SEATS(Employed ~ x11(seasonalma = "s3x5")),
  x11_6 = X_13ARIMA_SEATS(Employed ~ x11(seasonalma = "s3x9")))

x_11_decomps |>
  components() |> as_tsibble(key = .model, index = Month) |>
  filter(.model == "x11_1" | .model == "x11_2" | .model == "x11_3") |>
  autoplot(trend, size = 2, alpha = 0.7)

x_11_decomps |>
  components() |> as_tsibble(key = .model, index = Month) |>
  filter(.model == "x11_4" | .model == "x11_5" | .model == "x11_6") |>
  autoplot(seasonal, size = 2, alpha = 0.7)
```


16000
15000
14000
13000
trend
1990 Jan
2000 Jan
2010 Jan
2020 Jan
Month [1M]
.model
x11_1
x11_2
x11_3


seasonal
1.03
1.01
0.99
1990 Jan
2000 Jan
2010 Jan
2020 Jan
Month [1M]
.model
x11_4
x11_5
x11_6

- *X-11-ARIMA*, *X-12-ARIMA* and *X-13-ARIMA* are extensions of X11.

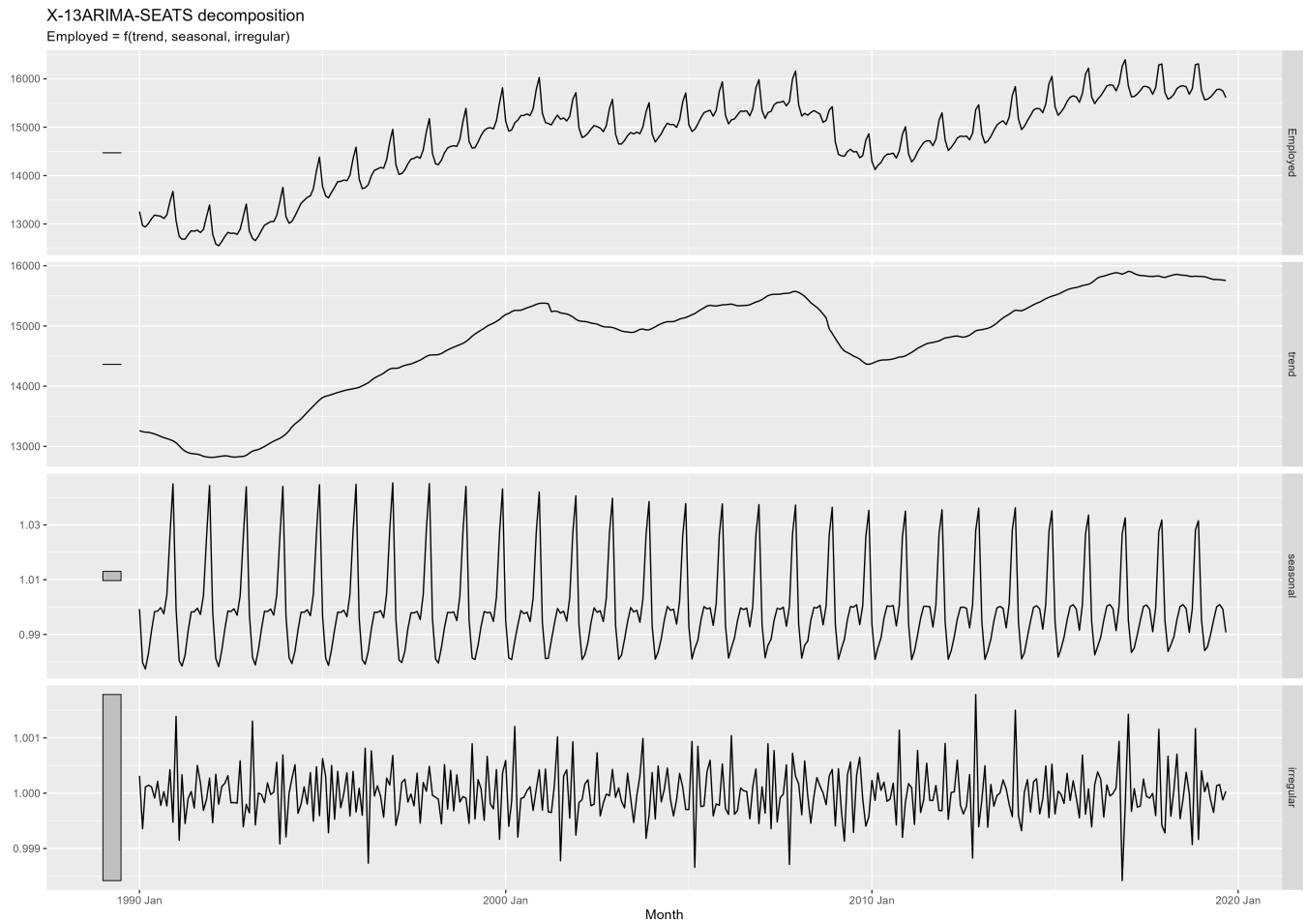
- The ARIMA part in their name refers to the fact that the ARIMA (forecasting) method is used to extend the time series and apply the X11 algorithm on the extended one (thus, eliminating the problem of having missing values at the end of the series for the trend estimation).

# SEATS method

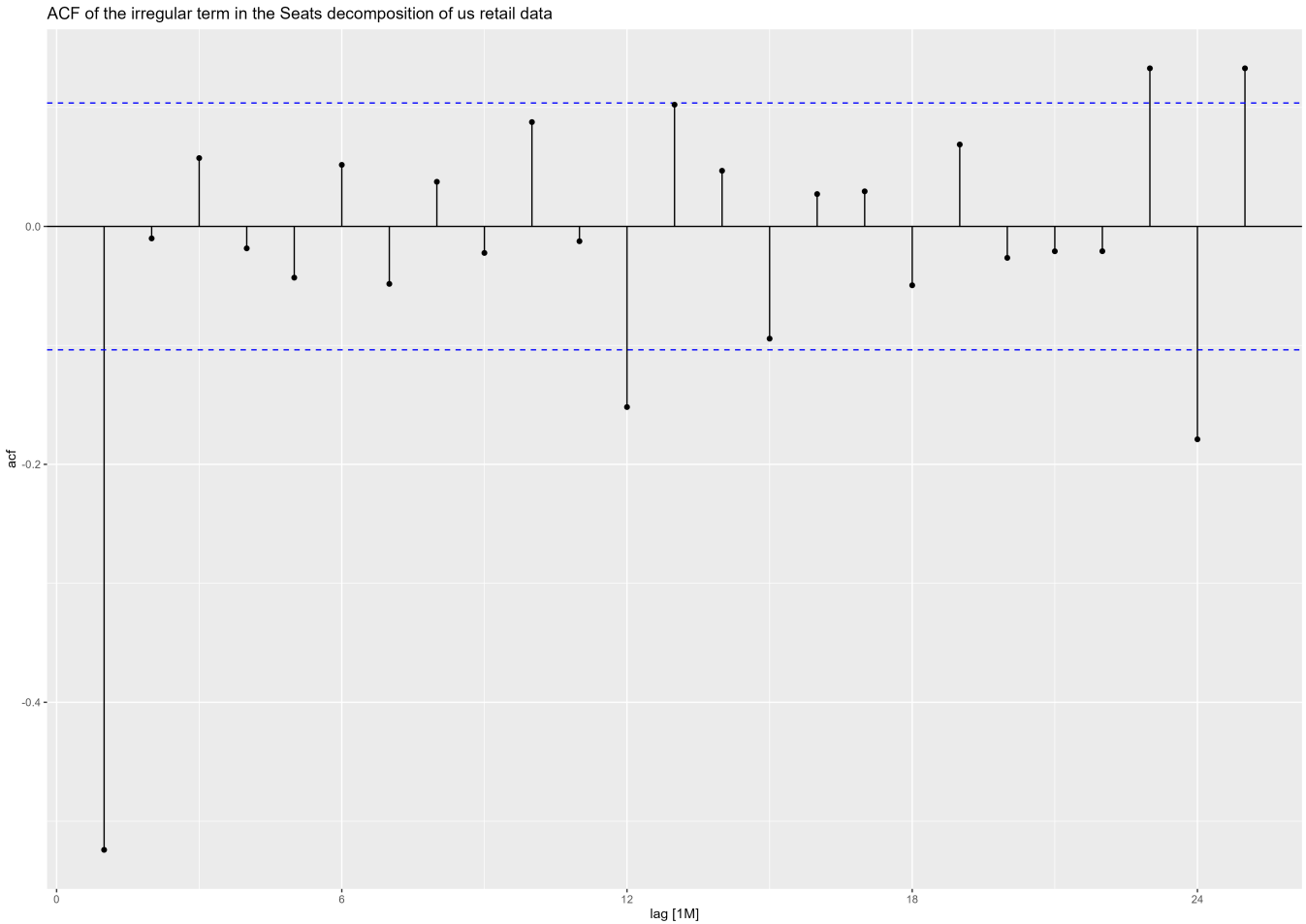
- Stands for "Seasonal Extraction in ARIMA Time Series"

```
seats_decomp <- us_retail_employment |>
  model(s = X_13ARIMA_SEATS(Employed ~ seats()))

seats_decomp |>
  components()

seats_decomp |>
  components() |>
  autoplot()

seats_decomp |>
  components() |>
  ACF(irregular) |>
  autoplot() +
  geom_point() +
  labs(title = "ACF of the irregular term in the Seats decomposition of us
      retail data")
```

```
# A dable: 357 x 7 [1M]
# Key:     .model [1]
# :        Employed = f(trend, seasonal, irregular)
   .model    Month Employed  trend seasonal irregular season_adjust
   <chr>     <mth>    <dbl>  <dbl>    <dbl>     <dbl>          <dbl>
 1 s       1990 Jan   13256. 13261.    0.999     1.00          13266.
 2 s       1990 Feb   12966. 13243.    0.980     0.999         13235.
 3 s       1990 Mar   12938. 13236.    0.977     1.00          13238.
 4 s       1990 Apr   13012. 13233.    0.983     1.00          13235.
 5 s       1990 May   13108. 13222.    0.991     1.00          13223.
 6 s       1990 Jun   13183. 13206.    0.998     1.00          13205.
 7 s       1990 Jul   13170. 13187.    0.999     1.00          13190.
 8 s       1990 Aug   13160. 13165.    1.00      1.00          13162.
 9 s       1990 Sep   13113. 13145.    0.998     1.00          13146.
10 s       1990 Oct   13185. 13129.    1.00      1.00          13126.
# i 347 more rows
# i Use `print(n = ...)` to see more rows
```




- Based on this plot, do you think SEATS is doing additive or multiplicative decomposition?


ACF of the irregular term in the Seats decomposition of us retail data
acf
0.0
-0.2
-0.4
0
6
12
18
24
lag [1M]

- SEATS allows estimates at the endpoints and it allows changing seasonality, but, similar to X11, it only works for quarterly and monthly data.

# STL decomposition I

- The name stands for "Seasonal and Trend decomposition using Loess" and it was developed in 1990.
- It uses a locally fitted regression model called Loess (Locally Estimated Scatterplot Smoothing), which is a generalization of moving averages and polinomial regression.
- It is an iterative process, first the series is detrended, then the seasonal component is estimated. After that the seasonal component is subtracted from the original data and the trend is estimated again, and so on. The Loess is used both for estimating the seasonal component and the trend.
- The remainder is what we have after the trend and the seasons were removed.

# STL decomposition II

- Advantages of STL:
  - Flexible: it can handle a wide range of time series, including ones where the seasonal pattern changes over time (and the rate of change can be controlled by the user). It can also handle any type of seasonality (weekly, monthly, quarterly, etc.)
  - Improved accuracy: it can provide a smoother estimate of the trend than the classical decomposition. Smoothness of the trend can also be controlled by the user.
  - Automatic parameter selection: the method automatically selects the best parameters
  - Customizability: on the other hand, through different parameters, it offers the possibility to fine-tune the decomposition according to the characteristics of the data.
  - Robust: user can specify that they want a robust decomposition, and then occasional unusual observations will not affect the trend and seasonal components (only the remainder will be affected).

# STL decomposition III

- Disadvantages of STL:
  - Can only do additive decomposition (for other types of time series transformations are needed).

# STL decomposition IV

- In R, there are two *specials* that can be set when using the STL decomposition: one for the trend and one for the seasonal component.
- For the trend the trend-cycle window size can be set, which is the number of consecutive observations to be used for estimating the trend. It needs to be an odd number and it controls how fast the trend can change. Larger values lead to smother trends. Default is 21.
- For the seasonal component the window size can be set, which is the number of consecutive years to be used in estimating each value in the seasonal component. It needs to be an odd number but it can be set to *periodic* which means we want identical seasons over the years. Default value is 13.

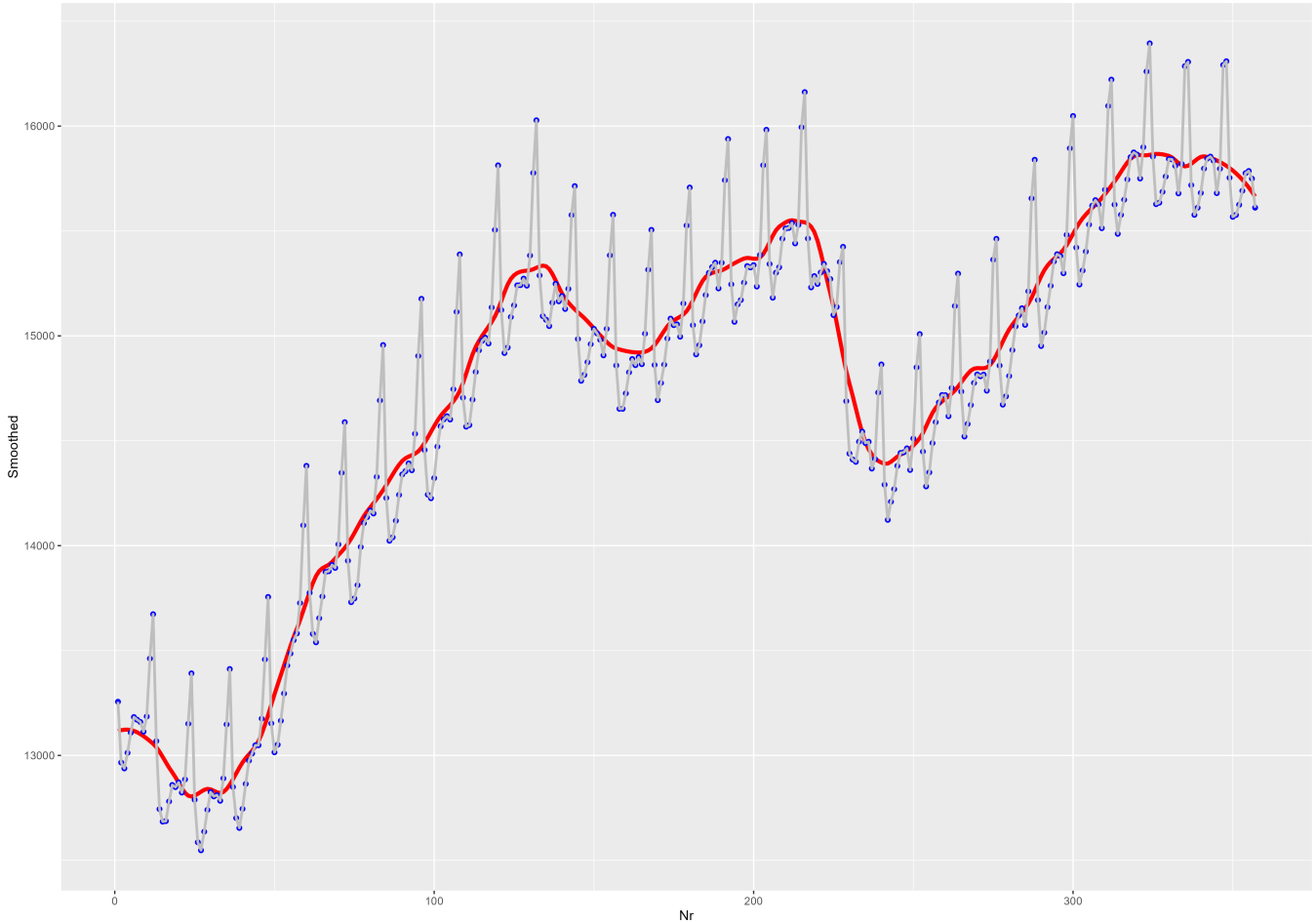
# What is LOESS?

- LOESS is a regression method, which fits a curve to a set of points, but this curve is built by fitting several models on subsets of the data (for every point, only points in its neighbourhood are considered).

- In R it is implemented in the *stats* package with the *loess* function.

```
us_retail_employment_index <- us_retail_employment |>
  mutate(Nr = seq(from = 1, to = 357, by= 1))
us_retail_employment_index |> view()
us_retail_employment_index |> autoplot(Employed)


loess_model <- loess( Employed ~ Nr, data = us_retail_employment_index, span =
     0.1)
loess_model
us_retail_employment_index <- us_retail_employment_index |>
  mutate(Smoothed = predict(loess_model))
us_retail_employment_index

us_retail_employment_index |> ggplot(mapping = aes(x = Nr)) +
  geom_line(mapping = aes(y = Smoothed), color = "red") +
  geom_point(mapping = aes(y = Employed), color = "blue") +
  geom_line(mapping = aes(y = Employed), color = "grey")
```

- Example of the LOESS curve fit on the Employment data.

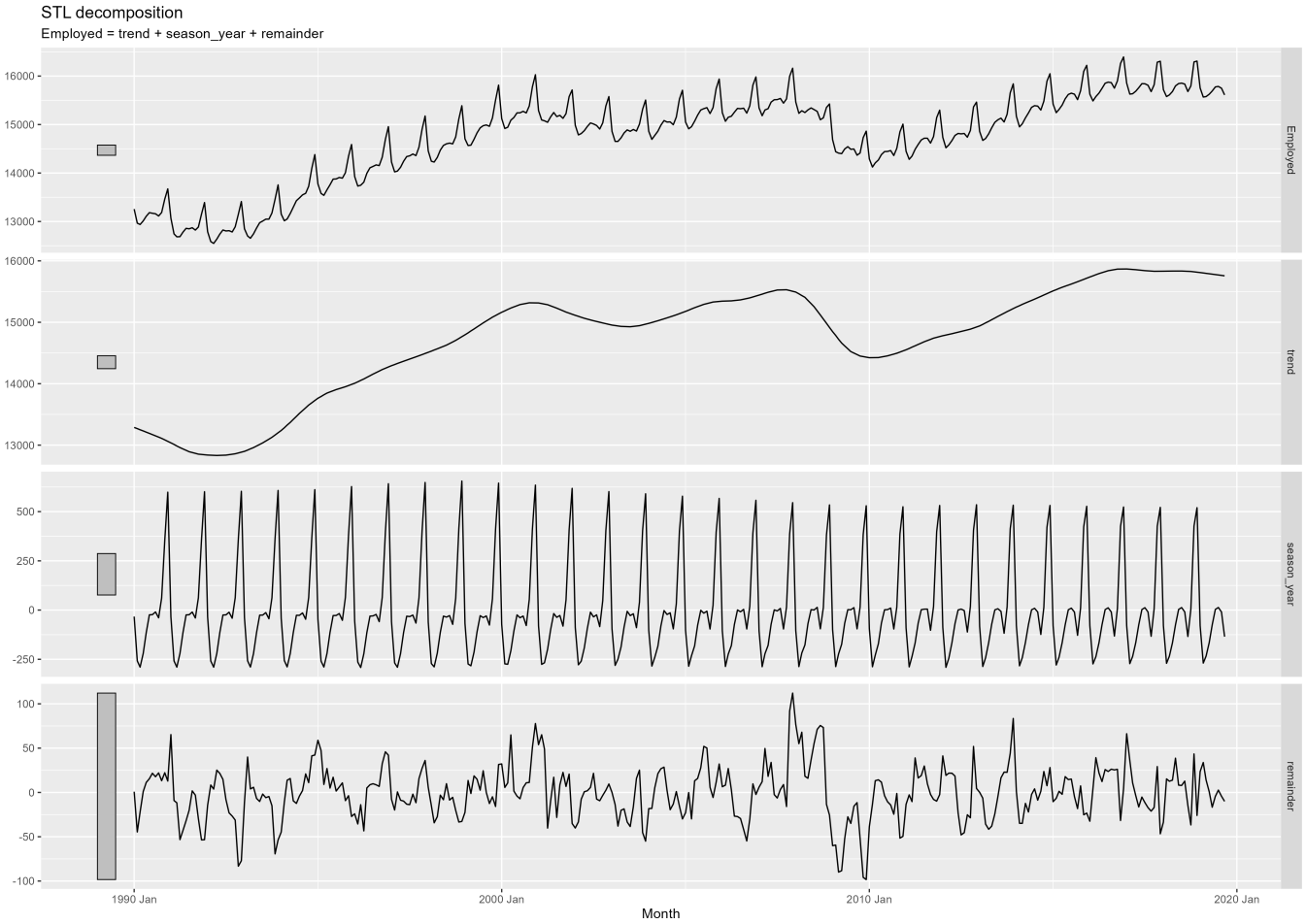


- Let us see a few examples of the STL decomposition for the US employment data with different parameter settings.

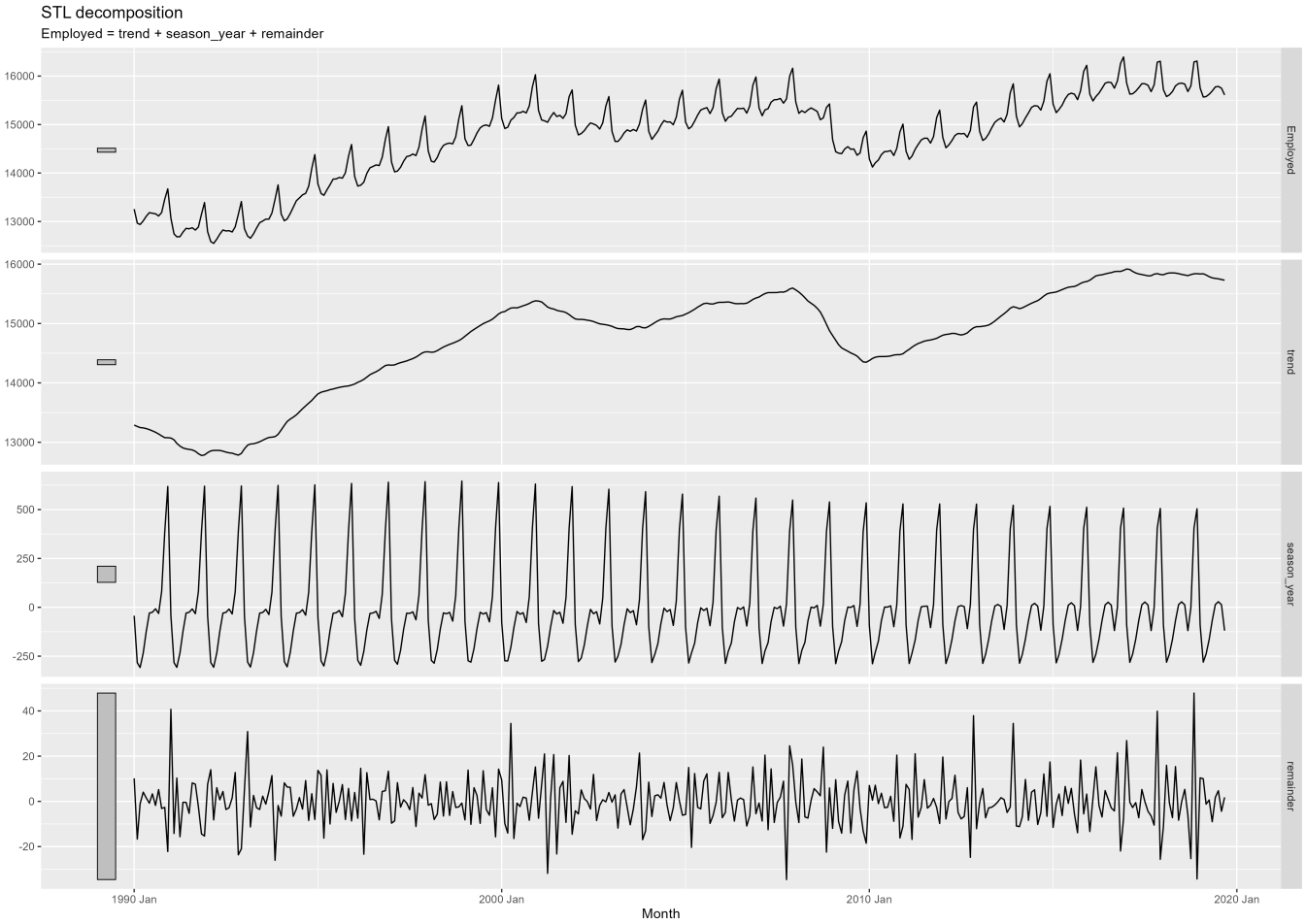
```
stl_dcm <- us_retail_employment |>
  model(Stl = STL(Employed)) |>
  components()
stl_dcm |> autoplot()

stl_dcm2 <- us_retail_employment |>
  model(Stl = STL(Employed ~ trend(window = 5))) |>
  components()
stl_dcm2 |> autoplot()

stl_dcm3 <- us_retail_employment |>
  model(Stl = STL(Employed ~ trend(window = 51))) |>
  components()
stl_dcm3 |> autoplot()

stl_dcm4 <- us_retail_employment |>
  model(Stl = STL(Employed ~ trend(window = 501))) |>
  components()
stl_dcm4 |> autoplot()

stl_dcm5 <- us_retail_employment |>
  model(Stl = STL(Employed ~ season(window = 5))) |>
  components()
stl_dcm5 |> autoplot()
```

```
stl_dcm6 <- us_retail_employment |>
  model(Stl = STL(Employed ~ season(window = 25))) |>
  components()
stl_dcm6 |> autoplot()

stl_dcm7 <- us_retail_employment |>
  model(Stl = STL(Employed ~ season(window = "periodic"))) |>
  components()
stl_dcm7 |> autoplot()
```

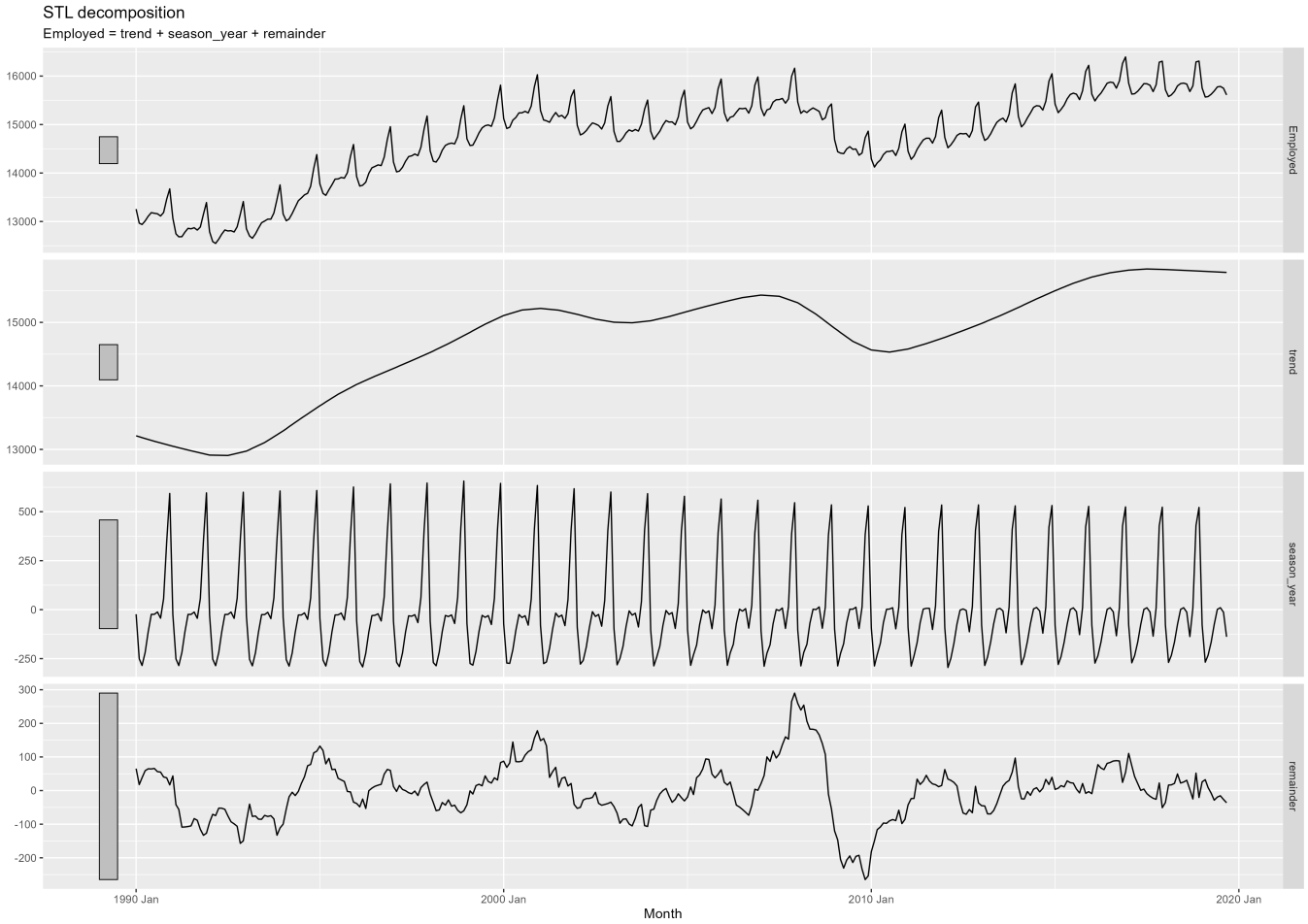
- STL decomposition with default parameters



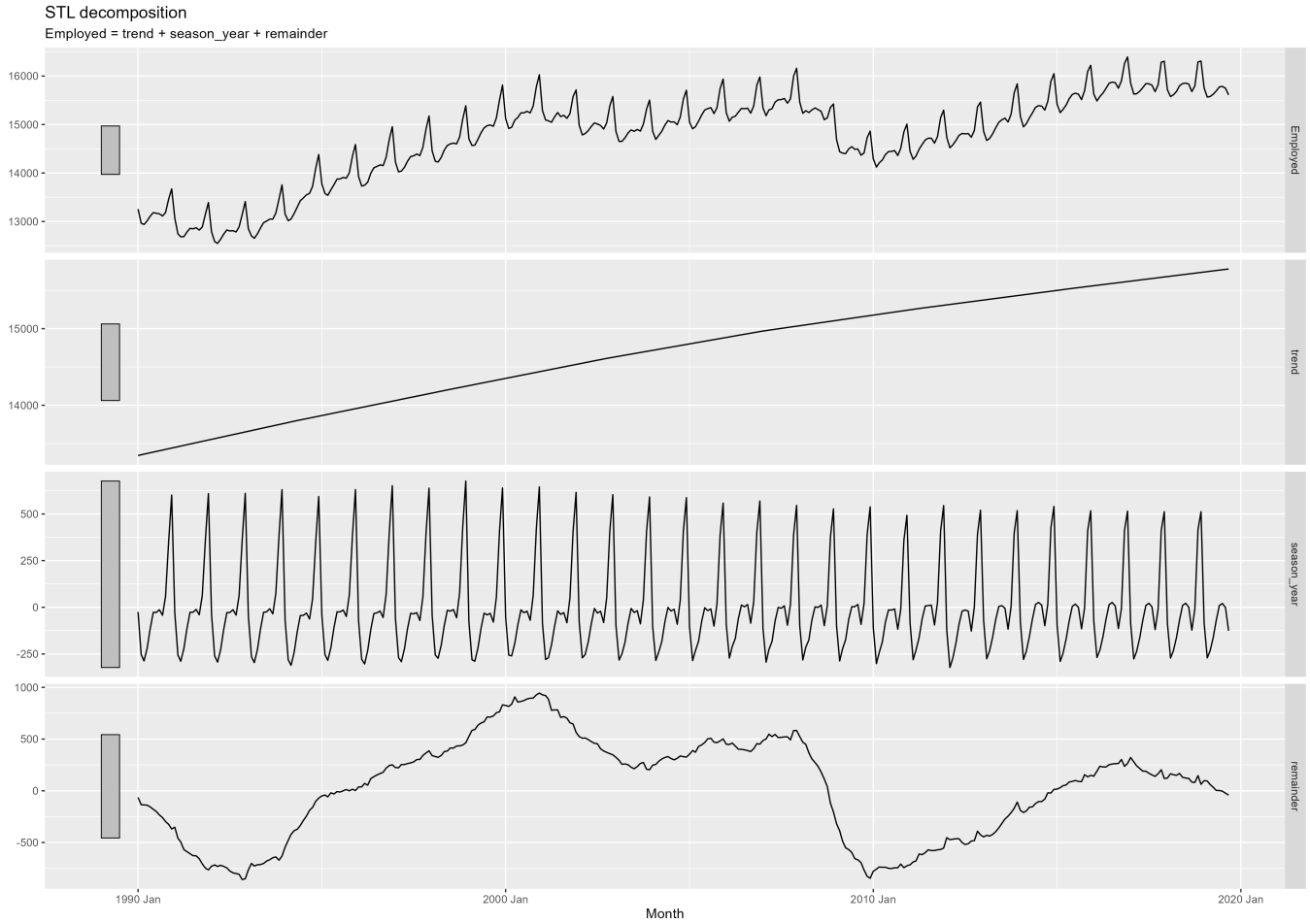
- STL decomposition with trend window = 5



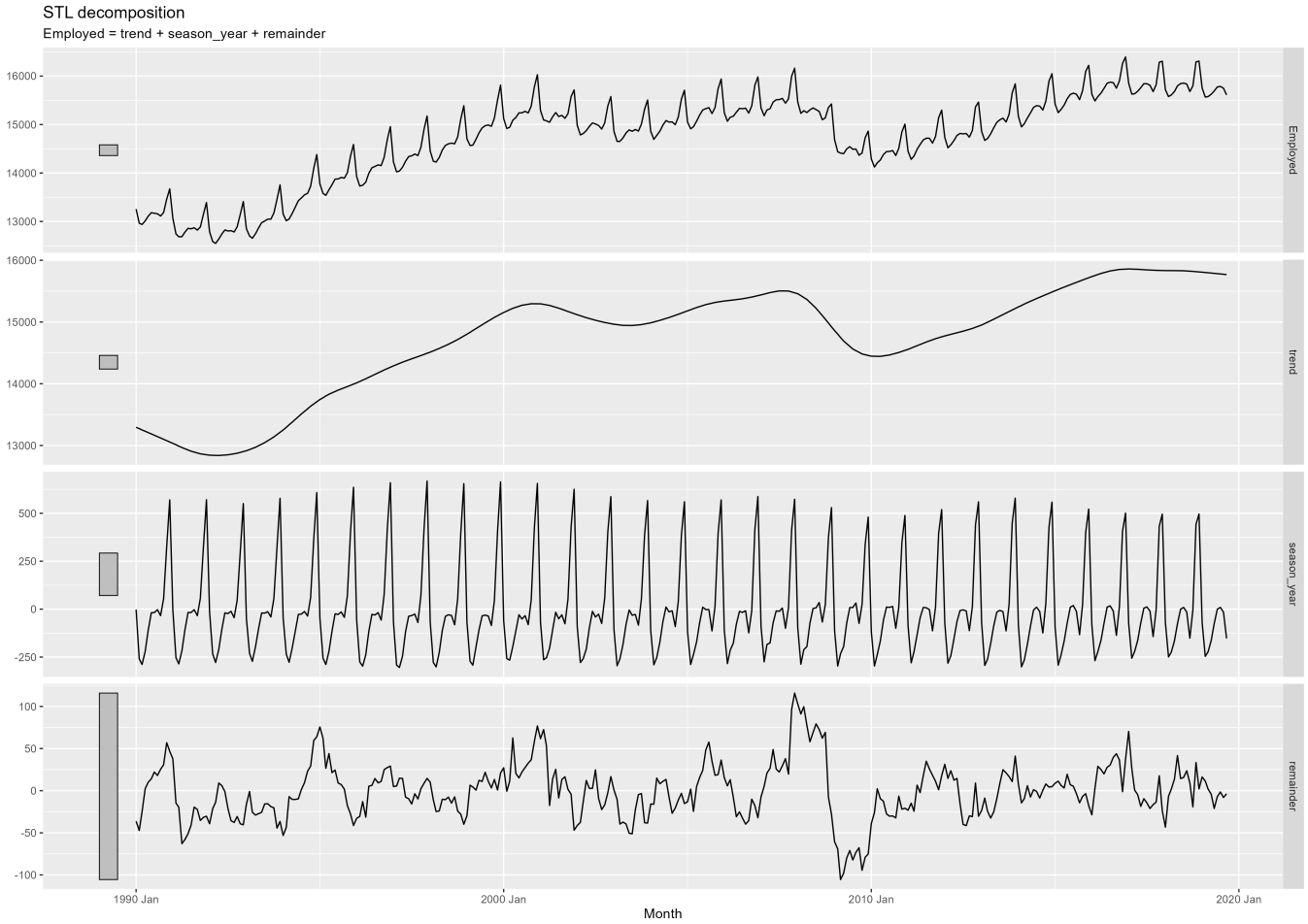
- STL decomposition with trend window = 51



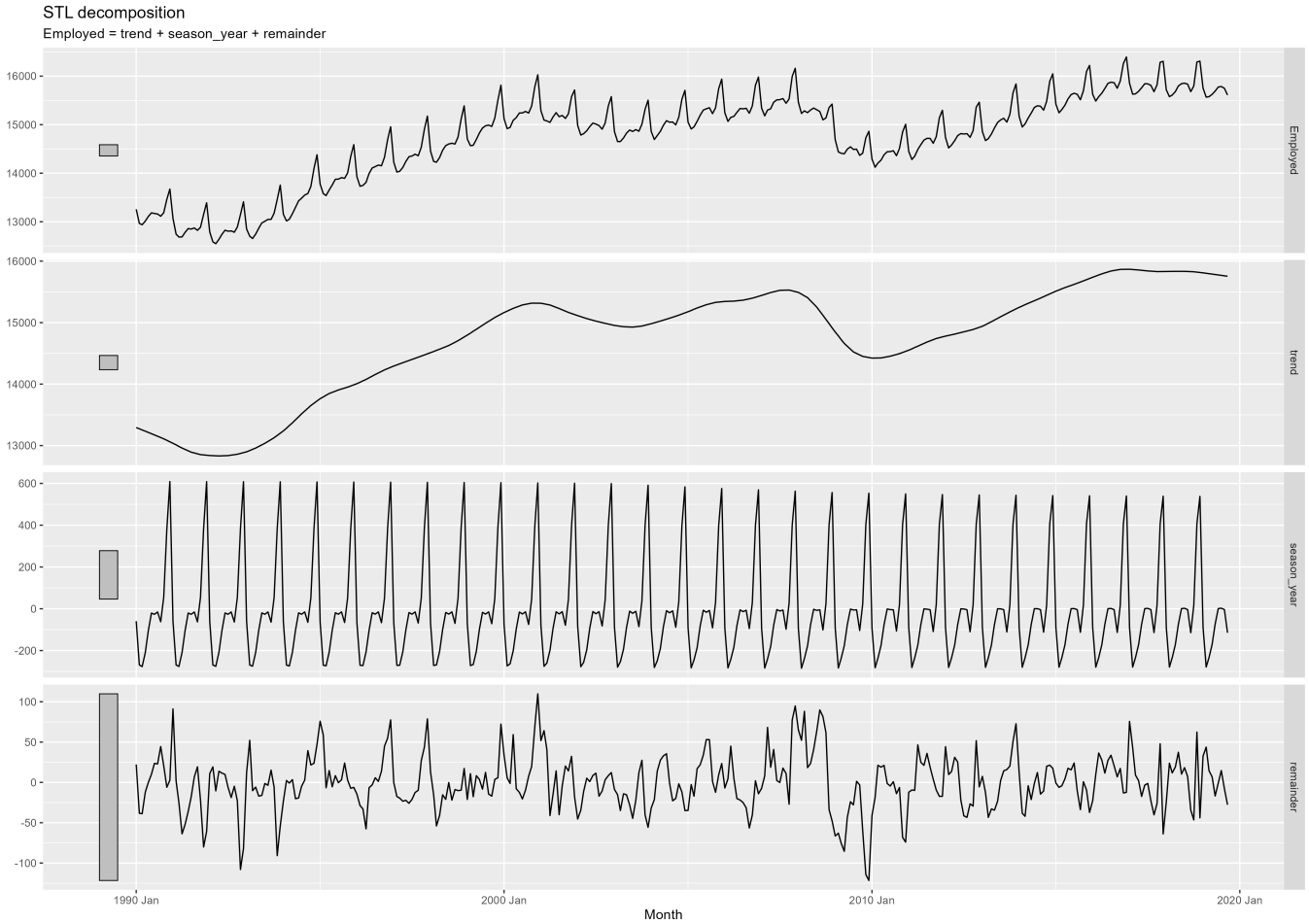
- STL decomposition with trend window = 501 (more than the number of observations)



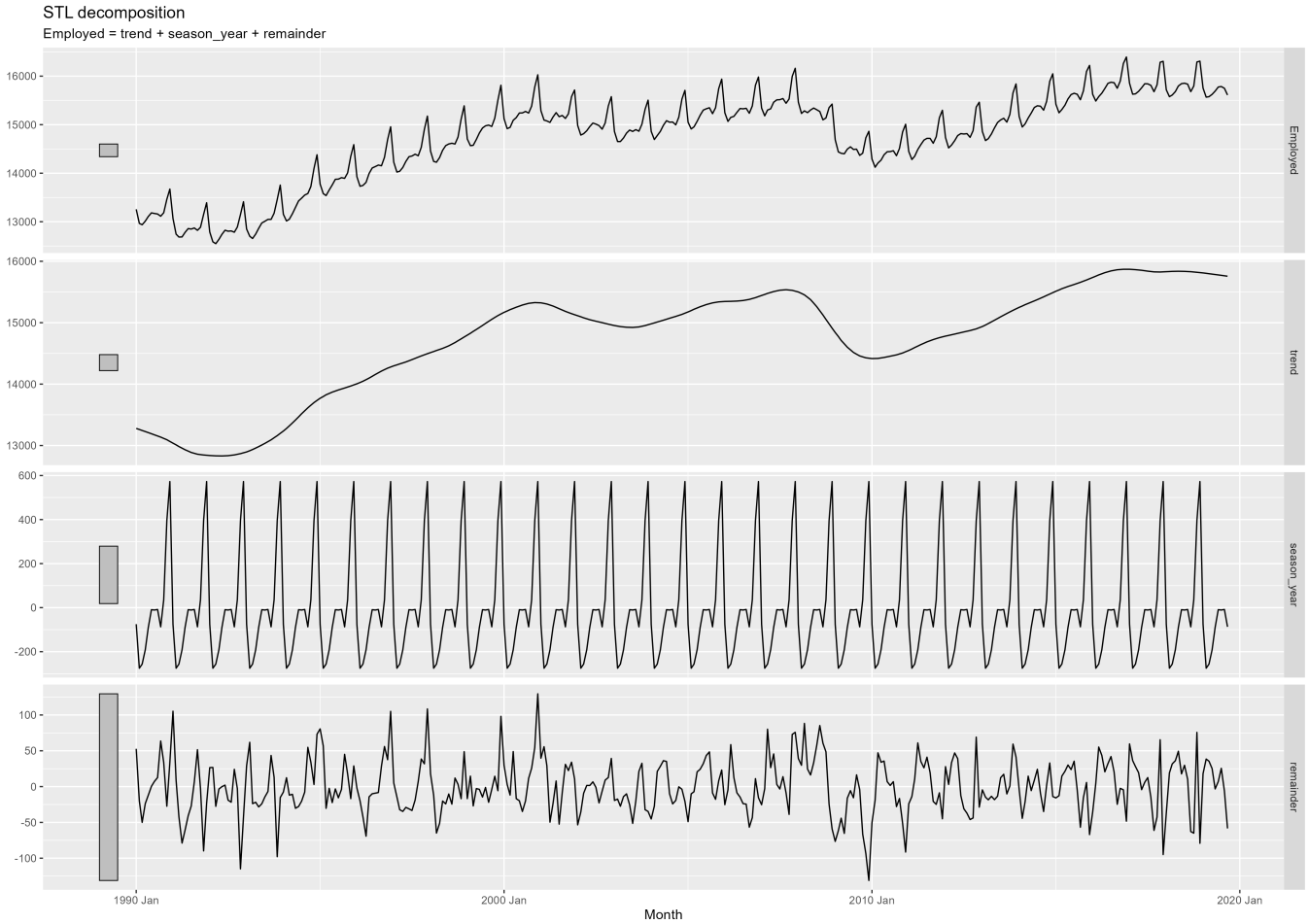
- STL decomposition with season window = 5



- STL decomposition with season window = 25



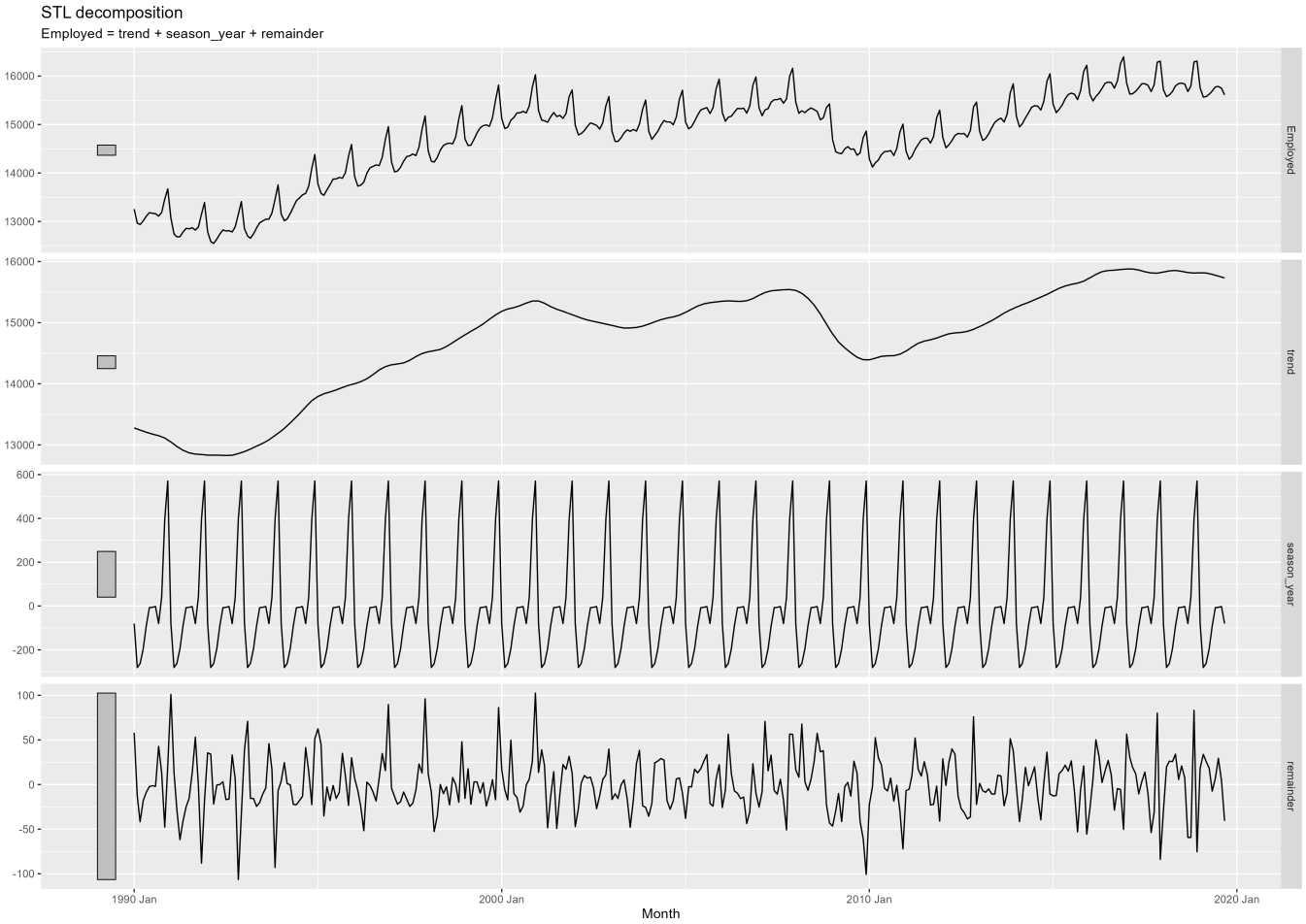
- STL decomposition with season window = "periodic"



- Obviously, we can specify both the trend and the season window in the same time.

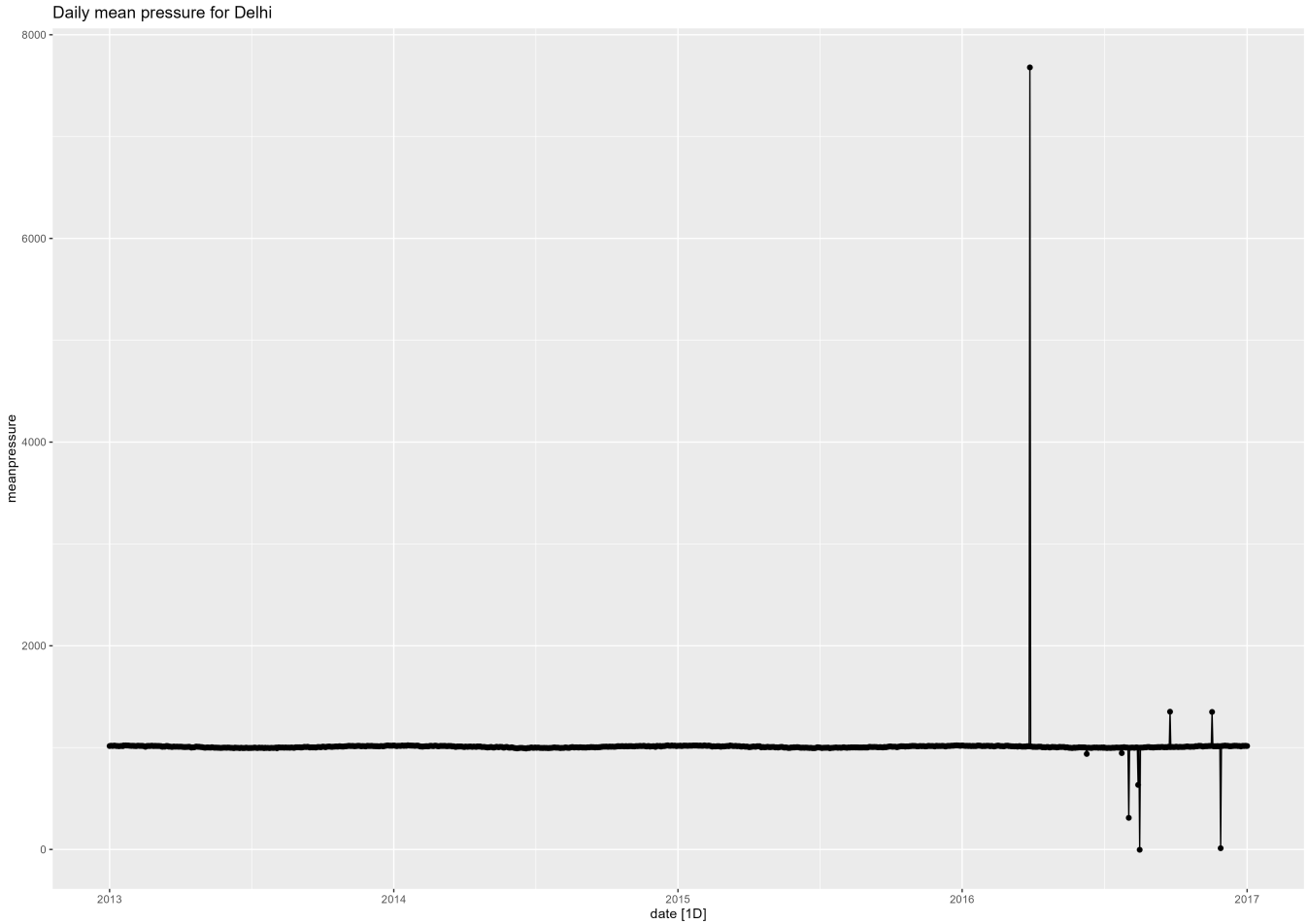
```
stl_dcm8 <- us_retail_employment |>
  model(Stl = STL(Employed ~ trend(window = 13) + season(window = "periodic")))
    |>
  components()
stl_dcm8 |> autoplot()
```

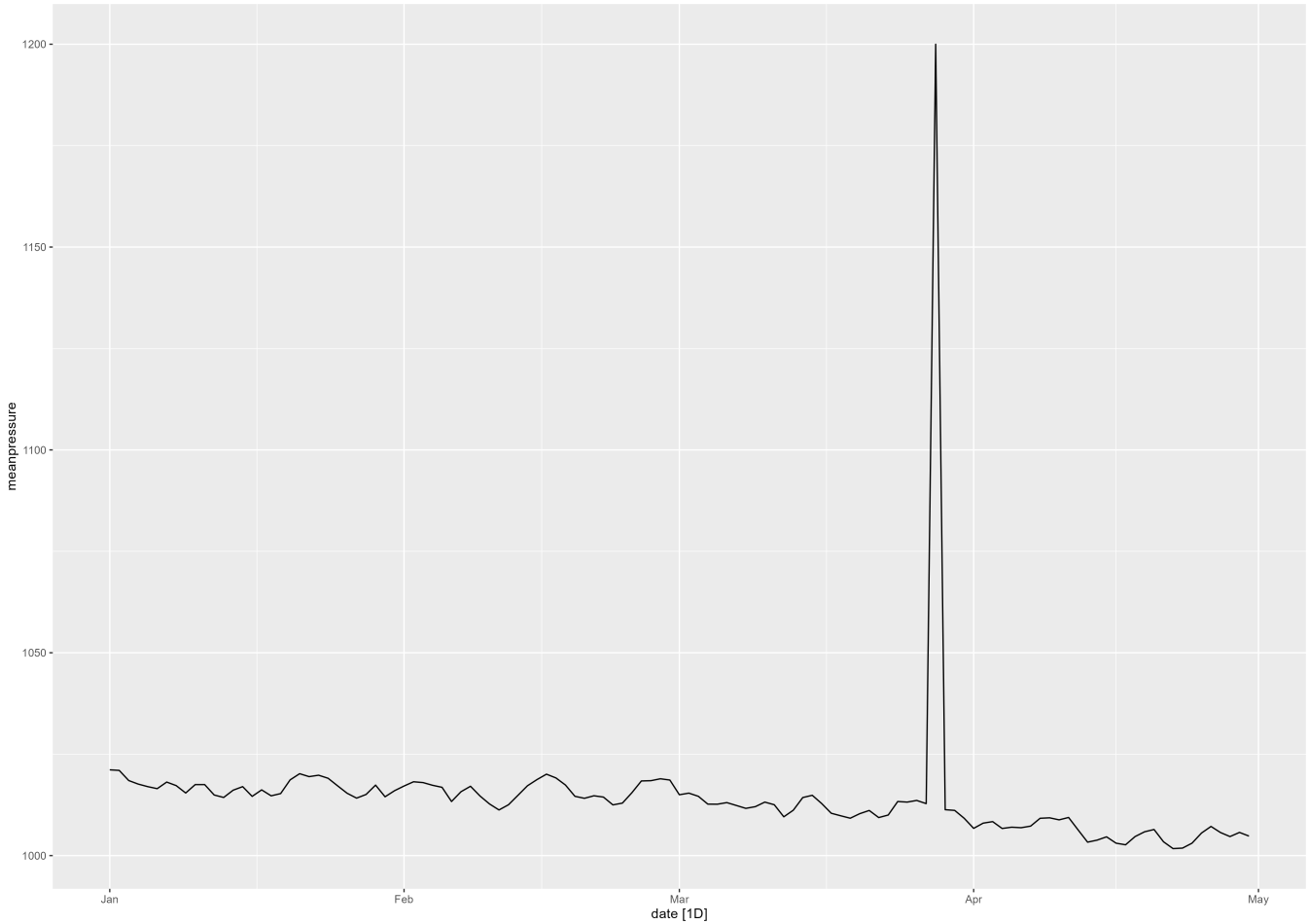
- Season window = periodic and trend window = 13



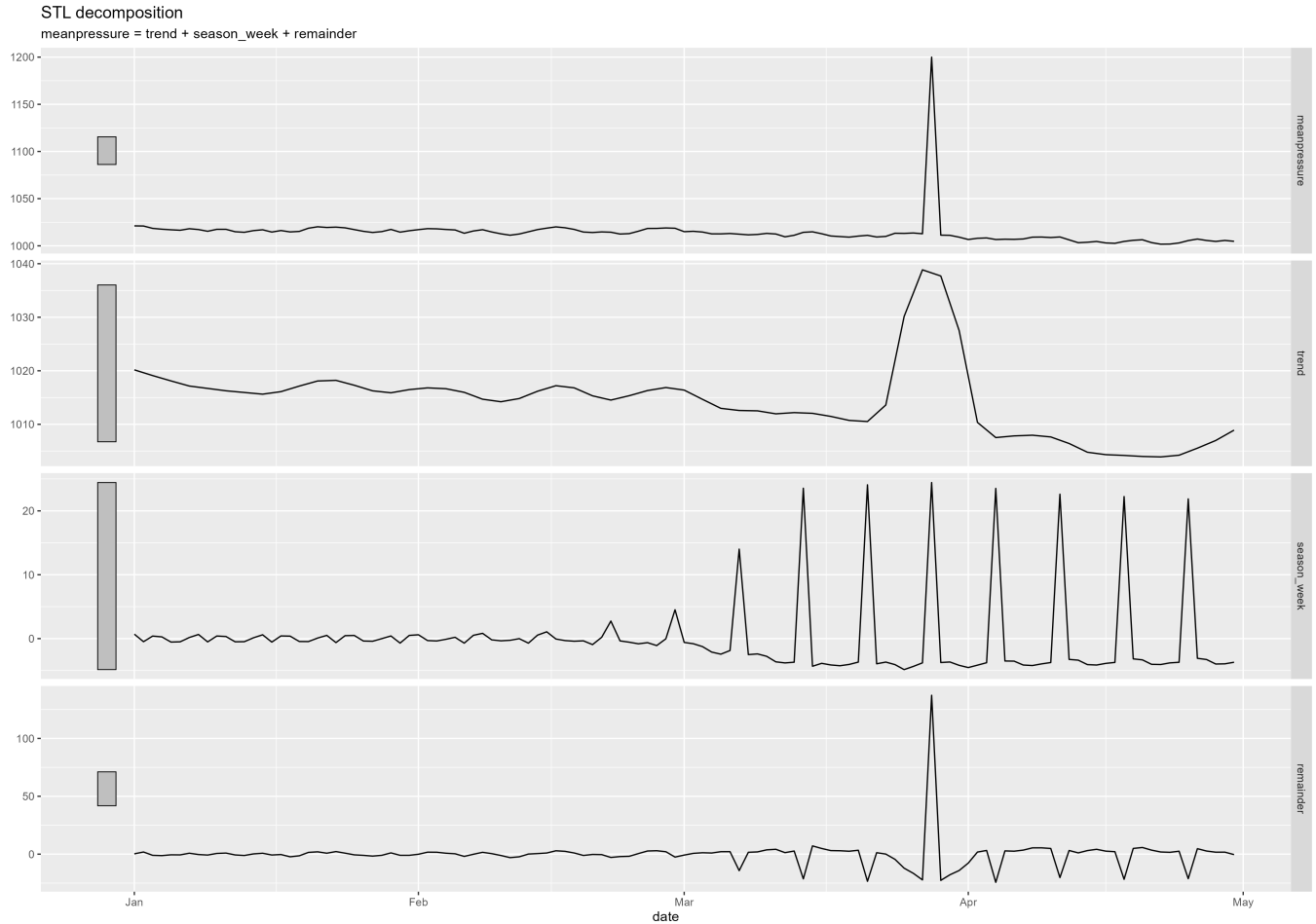
- There are two more parameters that can be set for the STL decompositon in R, which do not influence the trend or season directly: the number of iterations and whether to use a robust decomposition.

```
stl_climate <- climate_ts_short |>
  model(Stl = STL(meanpressure)) |> components()
stl_climate |> autoplot()

stl_climate2 <- climate_ts_short |>
  model(Stl = STL(meanpressure, robust = TRUE)) |> components()
stl_climate2 |> autoplot()
```

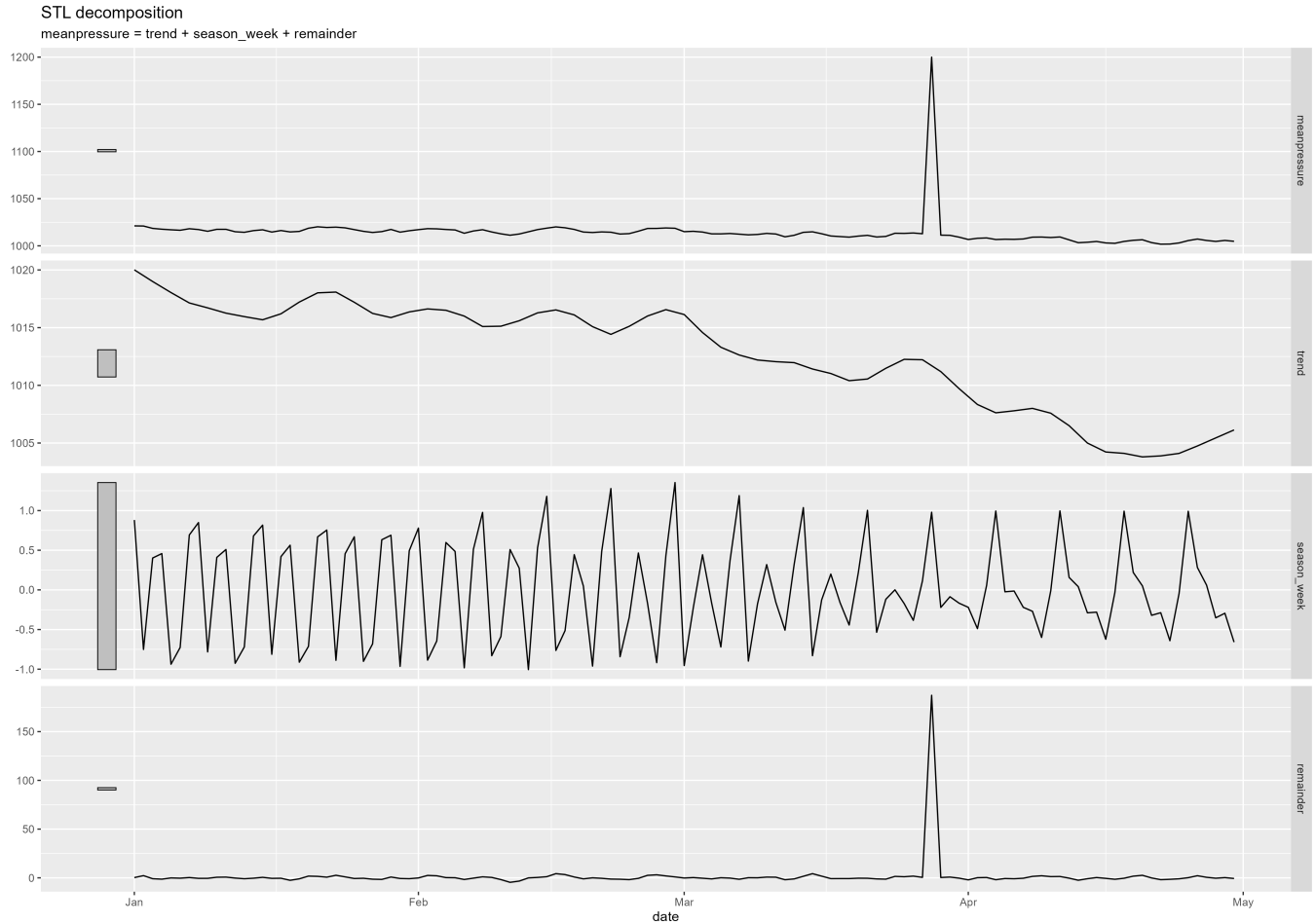

Daily mean pressure for Delhi
8000
6000
4000
2000
0
meanpressure
2013
2014
2015
2016
2017
date [1D]


1200
1150
1100
1050
1000
meanpressure
Jan
Feb
Mar
Apr
May
date [1D]

- Default STL decomposition



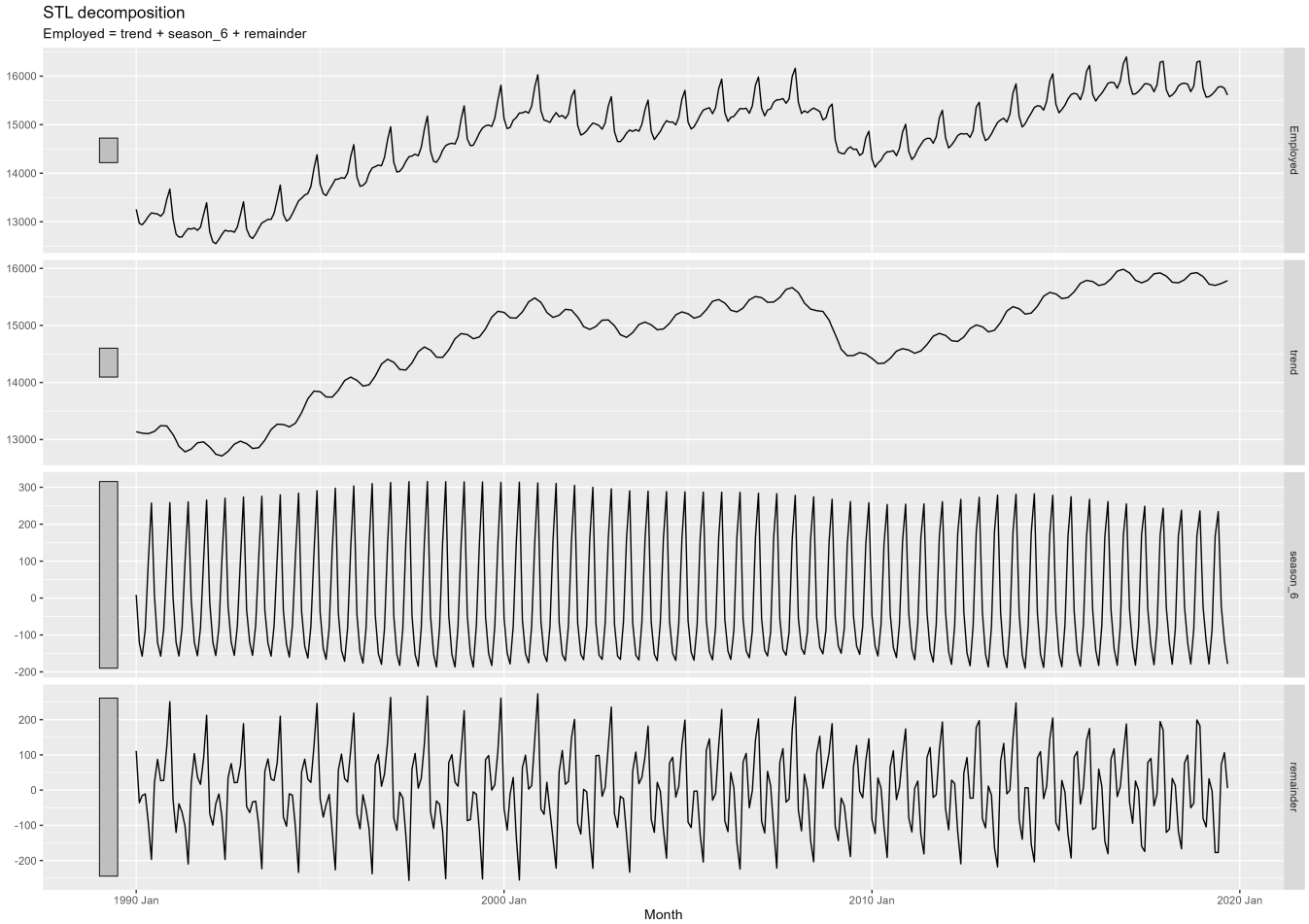
- Robust STL decomposition



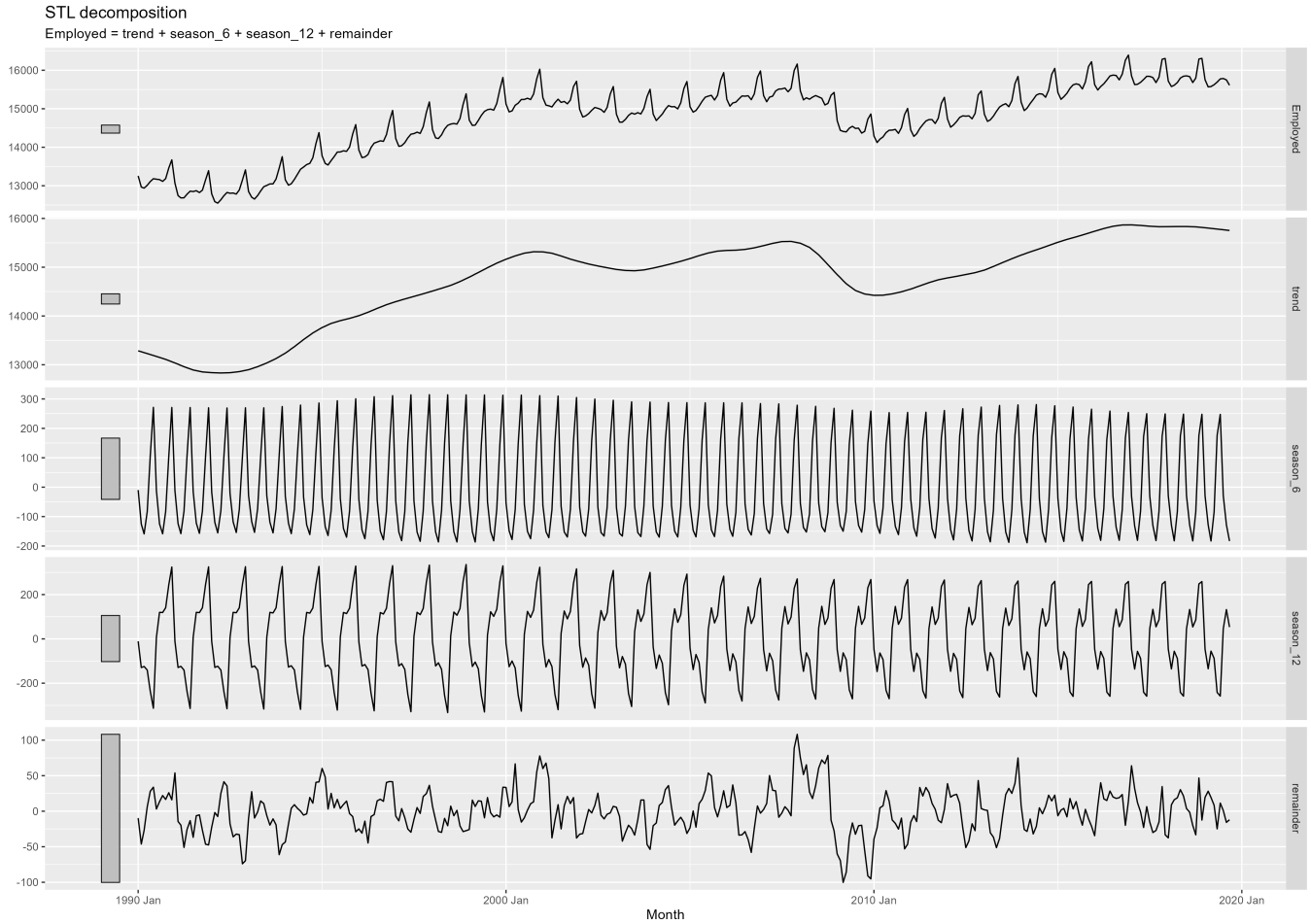
- For the seasonal special, besides the window size we can specify the period as well.
- We can also specify a list of periods.
- Let us see a few examples (for this time series, we do not have multiple seasonalities, just the yearly one, so the following examples have the goal of understanding the decomposition, this is not something you do in real life).

```
stl_dcm11 <- us_retail_employment |>
  model(stl = STL(Employed ~ season(period = 6))) |> components()
stl_dcm11 |> autoplot()
stl_dcm11 |> ACF(remainder) |> autoplot()

stl_dcm12 <- us_retail_employment |>
  model(stl = STL(Employed ~ season(period = c(6, 12)))) |> components()
stl_dcm12 |> autoplot()
```

- STL decomposition with a seasonal period of 6



- STL decomposition with a seasonal period of 6 and one period of 12



# Multiple Seasonal-Trend Decomposition using Loess (MSTL)

- It is an extension of the STL algorithm which can decompose data with multiple seasonalities.
- The method was proposed in: K. Bandara, R. J. Hyndman, C. Bergmeir: *MSTL: A Seasonal-Trend Decomposition Algorithm for Time Series with Multiple Seasonal Patterns*, 2021. `https://arxiv.org/abs/2107.13462`
- Since it can have several seasonal components, MSTL assumes that a time series can be decomposed as:

$$y_t = \hat{T}_t + \hat{S}_t^1 + \hat{S}_t^2 + \cdots + \hat{S}_t^n + R_t$$

- where the seasonal components represent different seasonalities (daily, weekly, monthly, etc.)

- The paper which introduced MSTL compared it to other existing methods capable of extracting multiple seasonalities and found that:
  - It is accurate (more accurate than the others on a set of benchmark time series)
  - Computationally efficient
  - Robust (by using the robust flag passed to STL).

# Short description of the MSTL algorithm I

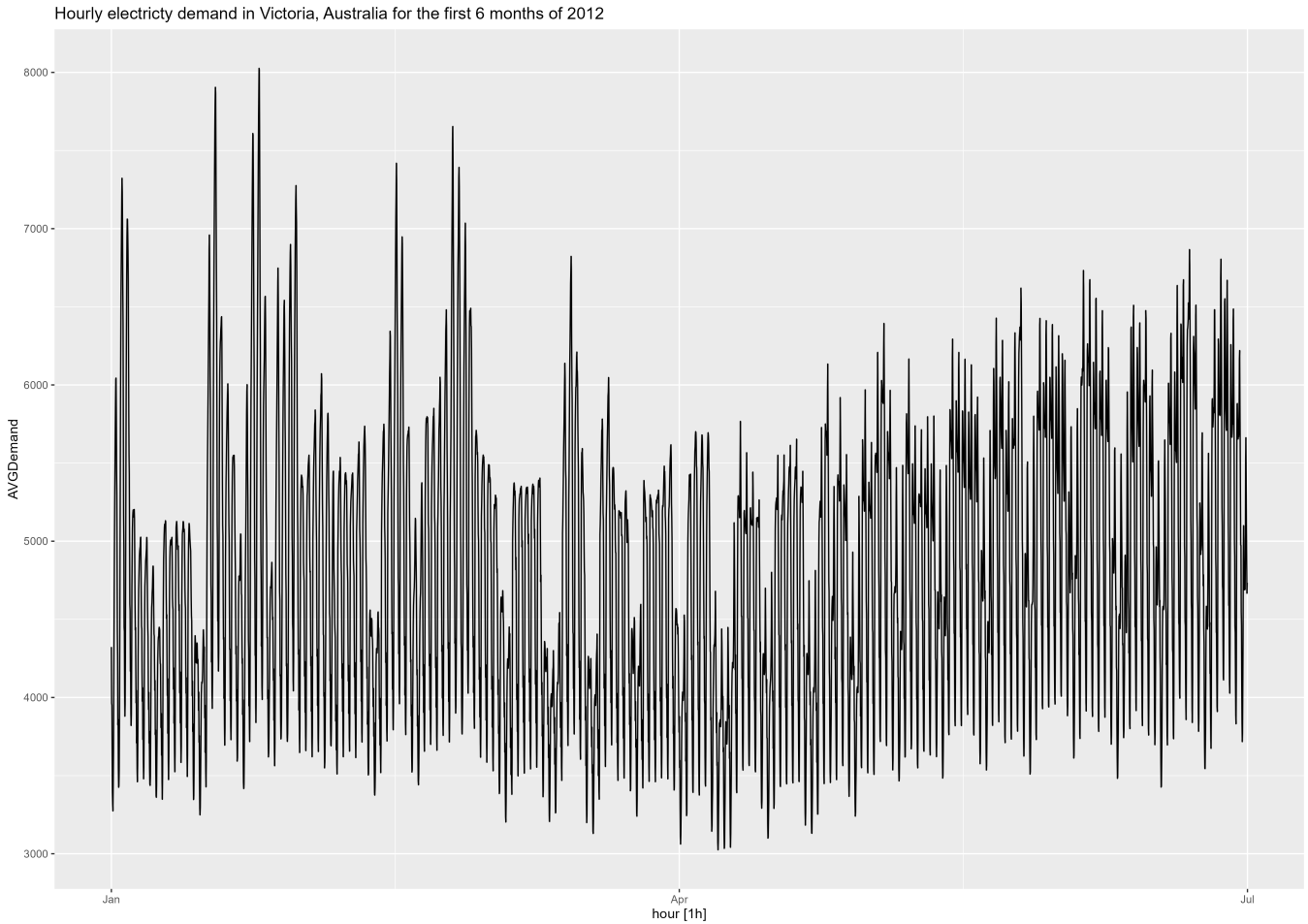
- MSTL receives as parameter the lengths of the seasonal components (the periods). For every period (starting from the shortest one and going to the longest one) we extract the seasonal components using STL and then subtract it from the time series.
- The seasonally adjusted time series from the previous step is then passed on, to continue extracting the next seasonal component (previous step is repeated with another period).
- At the end of the previous step we have estimates for a series of seasonalities and the fully seasonally adjusted time series (a time series from which all seasonal components were removed).

# Short description of the MSTL algorithm II

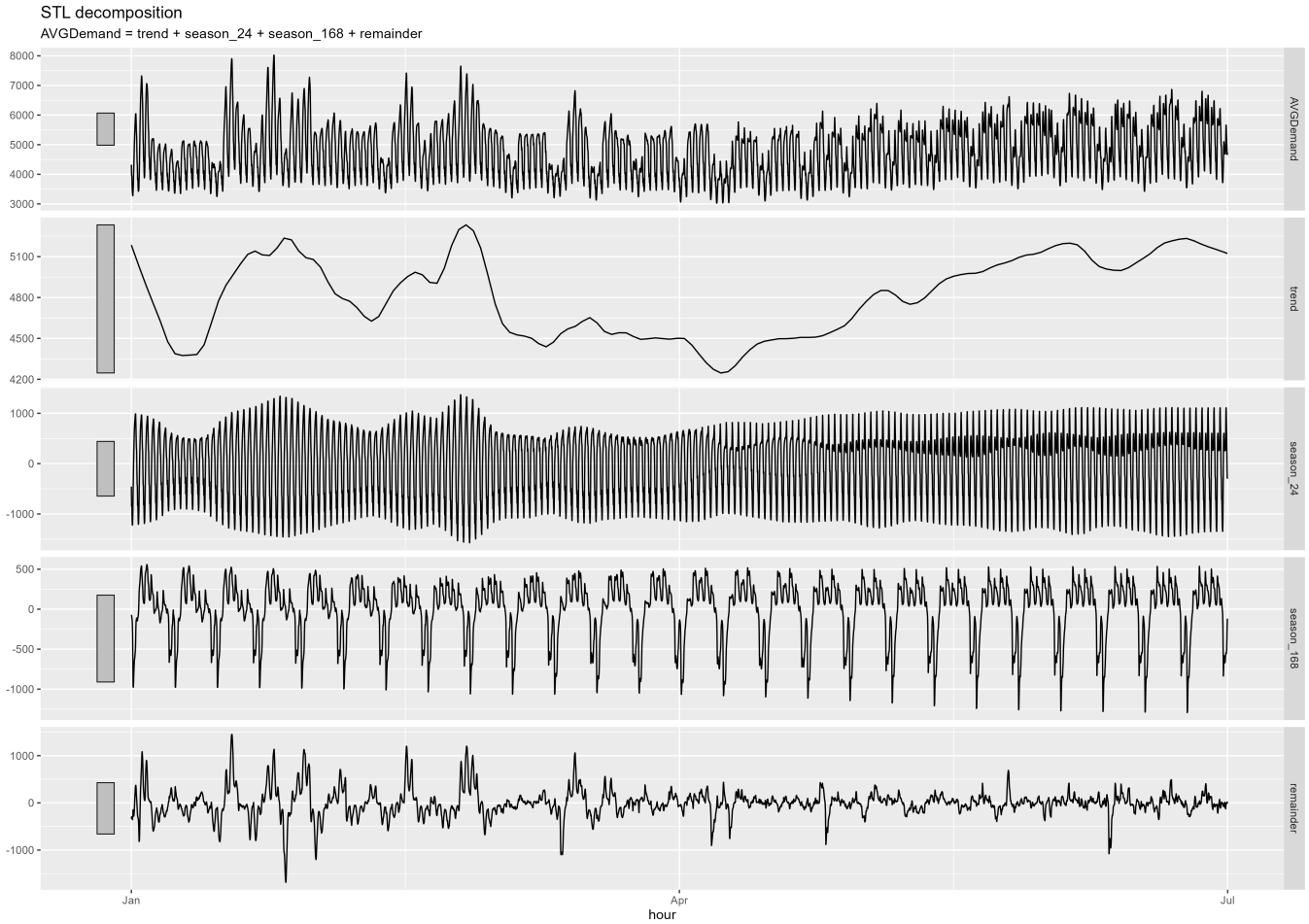
- In the next step we iterate through all seasonal components again, one-by-one, and at every iteration we add back that single seasonal component to the fully seasonally adjusted time series and re-extract it, using STL. Subtract it from the time series, to have again the fully de-seasonalised version.
- Repeat the previous step N times.
- Extract the trend from the last STL fit from the previous version (the one with the longest seasonal period component).
- Extract the residual by subtracting the trend from the de-seasonalised time series.

- The STL function from R that we have used previously, actually implements this MSTL algorithm.

```
vic_elec
vic_elec |>
  index_by(hour = floor_date(Time, "hour")) |>
  summarize(AVGDemand = mean(Demand)) -> vic_elec_hourly

vic_elec_hourly

vic_elec_hourly <- vic_elec_hourly |>
  filter(year(hour) == 2012, month(hour) <= 6)

vic_elec_hourly |>
  autoplot() +
  labs(title = "Hourly electricty demand in Victoria, Australia for the first 6
      months of 2012")

vic_elec_hourly |>
  model(ms = STL(AVGDemand ~ season(period = c(24, 24 * 7)))) |>
  components() |>
  autoplot()
```


Hourly electricty demand in Victoria, Australia for the first 6 months of 2012
AVGDemand
8000
7000
6000
5000
4000
3000
Jan
Apr
Jul
hour [1h]

- MSTL decomposition for VicElec data.



# Time series features I

- A feature is simply a value which describes some property of the time series.
- We can compute different features for time series which can help us explore the properties of the time series.
  - We have already seen some features (and we have actually used the *features* function) before: the Guerrero estimate for the Box-Cox transformations, the number of times regular or seasonal differencing should be applied to make a time series stationary, etc.
- For exploring different features, we will use as example the Australian quarterly tourism data, found in the *tourism* tsibble.

# Time series features II

- We have simple features: minimum, maximum, mean. The *features* function will compute them. In case of tsibbles containing several time series, it will compute the value for each time series.
- Function *quantile* provides the five summary statistics: minimum, first quartile, median, third quartile and maximum.

```
tourism |>
  features(Trips, list(Mean = mean)) |>
  arrange(Mean) #to sort increasingly by mean

tourism |>
 features(Trips, quantile)
```

# Autocorrelation features I

- Remember, autocorrelation is the correlation between a time series and its lagged version (version with some time steps behind).
- We have talked about the *acf* plot, where autocorrelation values for different lags are visualized.
- But we can compute new numerical features from autocorrelation, for example, the sum of squares for the first 10 autocorrelations is an indicator of how much autocorrelation is in the data in general.
- We can also difference the data and compute autocorrelation in this new series (both for simple and seasonal differences).
- We can difference the already differenced data as well and compute autocorrelation features for this as well.

# Autocorrelation features II

- In R we can compute autocorrelation features with the *feat_acf* function which returns the following:
  - the first autocorrelation coefficient of the original data
  - the sum of the squares for the first 10 autocorrelation coefficients of the original data (shows overall how much autocorrelation is in the data).
  - the first autocorrelation coefficient of the differenced data
  - the sum of the squares for the first 10 autocorrelation coefficients of the differenced data
  - the first autocorrelation coefficient of the twice differenced data
  - the sum of the squares for the first 10 autocorrelation coefficients of the twice differenced data
  - for seasonal data the autocorrelation coefficient at the first seasonal lag is also returned

# Autocorrelation features III

```
tourism |>
    features(Trips, feat_acf)
```

# Features based on the STL decomposition

- Remember, the STL decomposition means to split the time series into three components: trend, seasonal and remainder. STL assumes that there is an additive decomposition: $y_t = T_t + S_t + R_t$
- We can measure how strong the trend in the data is, using the following formula:

$$F_T = max(0, 1 - \frac{Var(R_t)}{Var(T_t + R_t)}$$

- where *Var* represents the variance of the data. $T_t + R_t$ represents the seasonally adjusted data (where the seasonal component is removed). If the data is strongly trended, the variation of the seasonally adjusted data should be a lot larger than the variation in the remainder (and $F_T$ is close to 1), while in case of data with little or no trend the two variations are similar.

- We can define the strength of the seasonality in the same manner, but this time using the detrended data (the data from which the trend component is removed)

$$F_S = max(0, 1 - \frac{Var(R_t)}{Var(S_t + R_t)}$$

- A value of $F_S$ close to 0 means no seasonality.
- $F_T$ and $F_S$ can be useful, when we have a lot of time series and we need to find the ones with the most trend or the most seasonality. The function *feat_stl* will compute these two features, together with some other features.

```
tourism |>
  features(Trips, feat_stl)

tourism |>
  features(Trips, feat_stl) |> arrange(desc(trend_strength))
```

- We can check how trended and seasonal are these time series by plotting trend strength against seasonal strength.

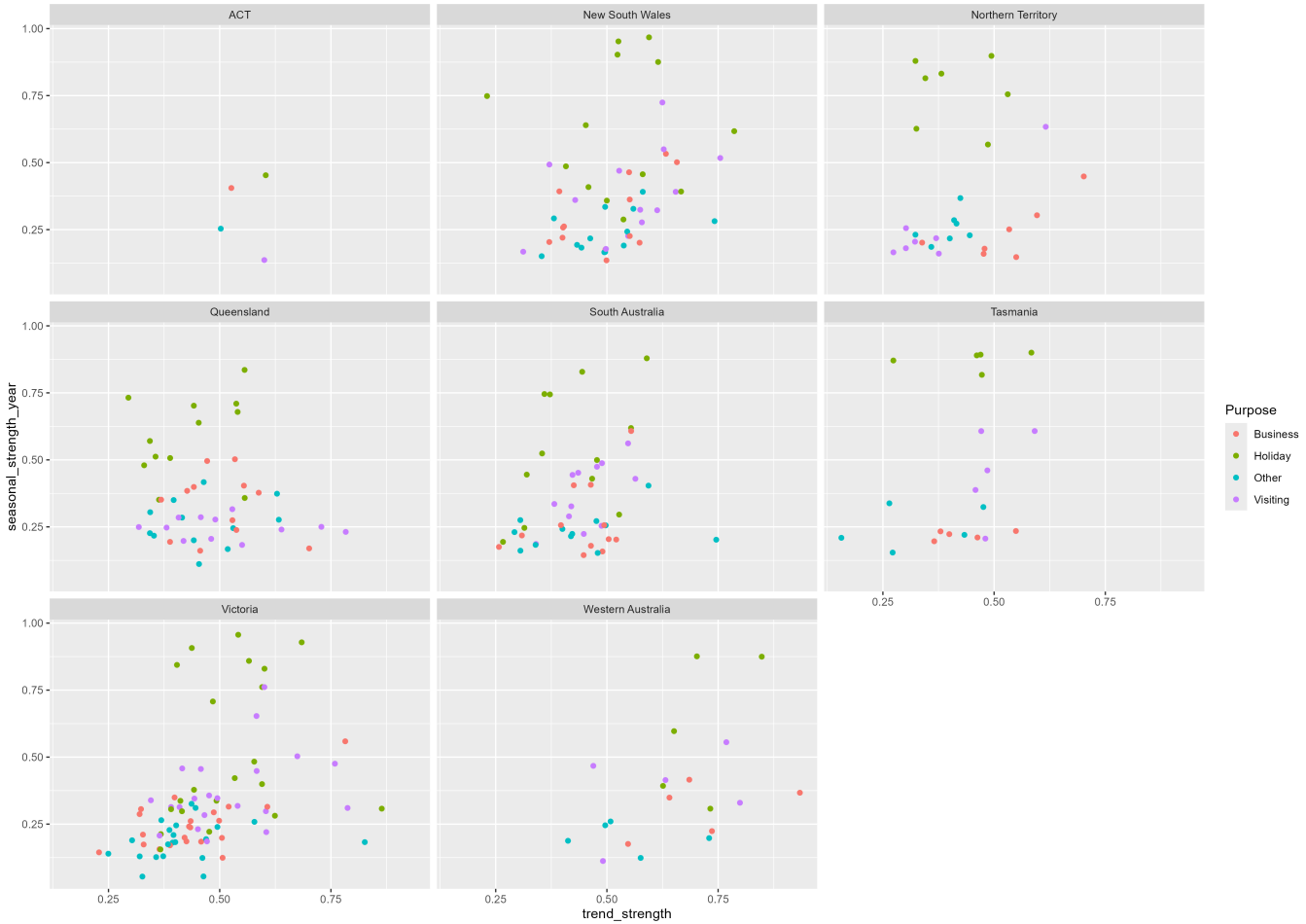
```
tourism |>
 features(Trips, feat_stl) |>
 ggplot(aes(x = trend_strength, y = seasonal_strength_year, col = Purpose)) +
 geom_point() +
 facet_wrap(vars(State))
```

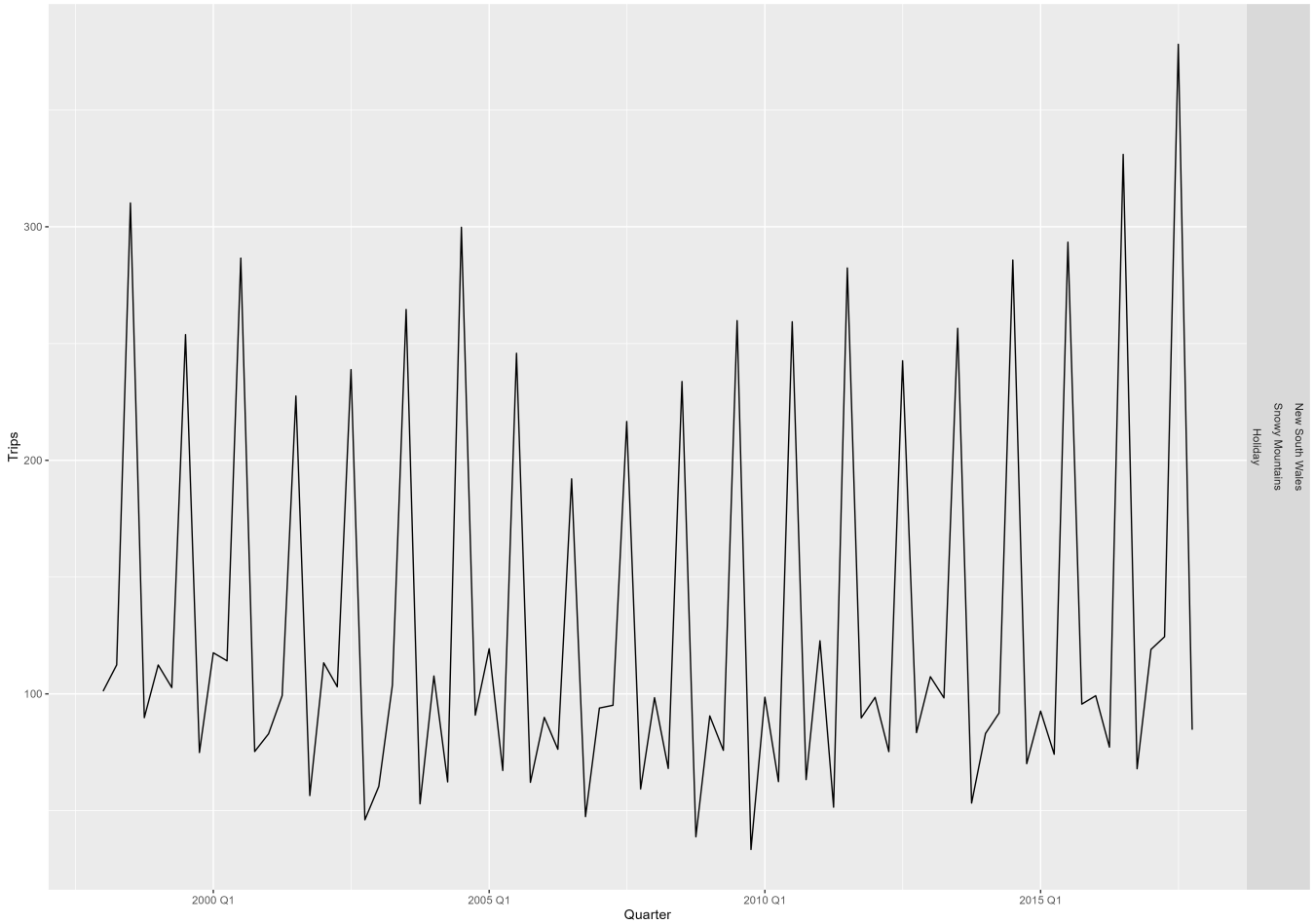
- Find the series with the strongest seasonal component

```
tourism |>
  features(Trips, feat_stl) |>
  filter(seasonal_strength_year == max(seasonal_strength_year)) |>
  left_join(tourism, by = c("State", "Region", "Purpose"), multiple = "all") |>
  ggplot(aes(x = Quarter, y = Trips)) +
  geom_line() +
  facet_grid(vars(State, Region, Purpose))
```

# Other STL features

- *seasonal_peak_year* - which month or quarter contains the largest values (the peak of the season). For example, in tourism data, if the peak is Q1, people are travelling more in winter (except if we are talking about Australia, because Q1 is summer there).
- *seasons_through_year* - which month or quarter contains the lowest values
- *spikiness* - measures the prevalence of spikes in the remainder
- *linearity* - how linear the trend component is
- *curvature* - measures the curvature of the trend component
- *stl_e_acf1* - the first autocorrelation value of the remainder
- *stl_e_acf10* - the sum of the squares of the first 10 autocorrelation features


ACT
New South Wales
Northern Territory
Queensland
South Australia
Tasmania
Victoria
Western Australia
seasonal_strength_year
trend_strength
1.00
0.75
0.50
0.25
Purpose
Business
Holiday
Other
Visiting


Trips
300
200
100
2000 Q1
2005 Q1
2010 Q1
2015 Q1
Quarter
New South Wales
Snowy Mountains
Holiday

- All features can be computed using the *feature_set* function, with the *pkgs* parameter set to "feasts". This will give us a total of 48 new features for each time series.

```
tourism_features <- tourism |>
  features(Trips, feature_set(pkgs = "feasts"))
view(tourism_features)
```